

Desenvolvimento de aplicações com IA

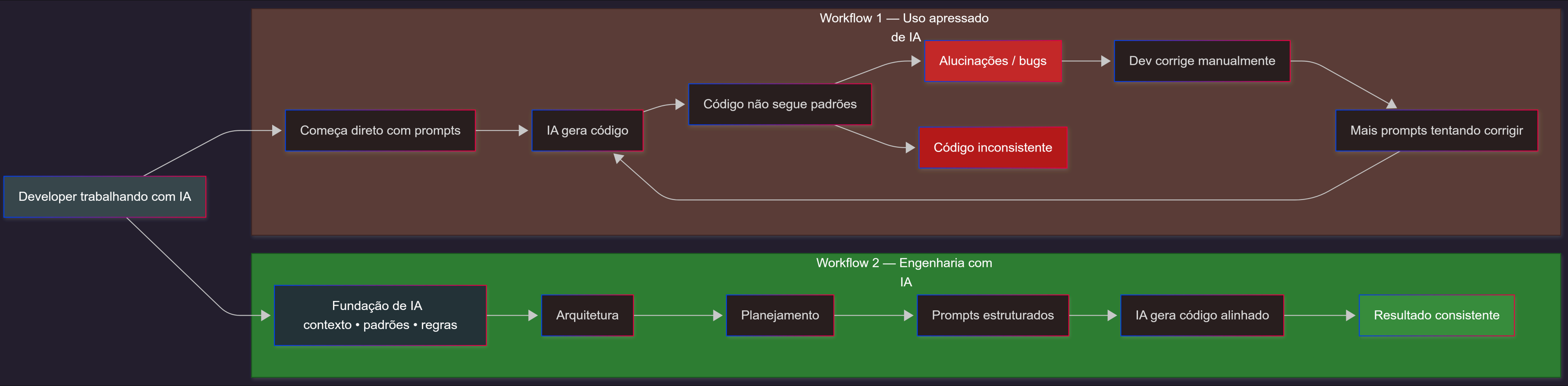
Arquitetura do projeto



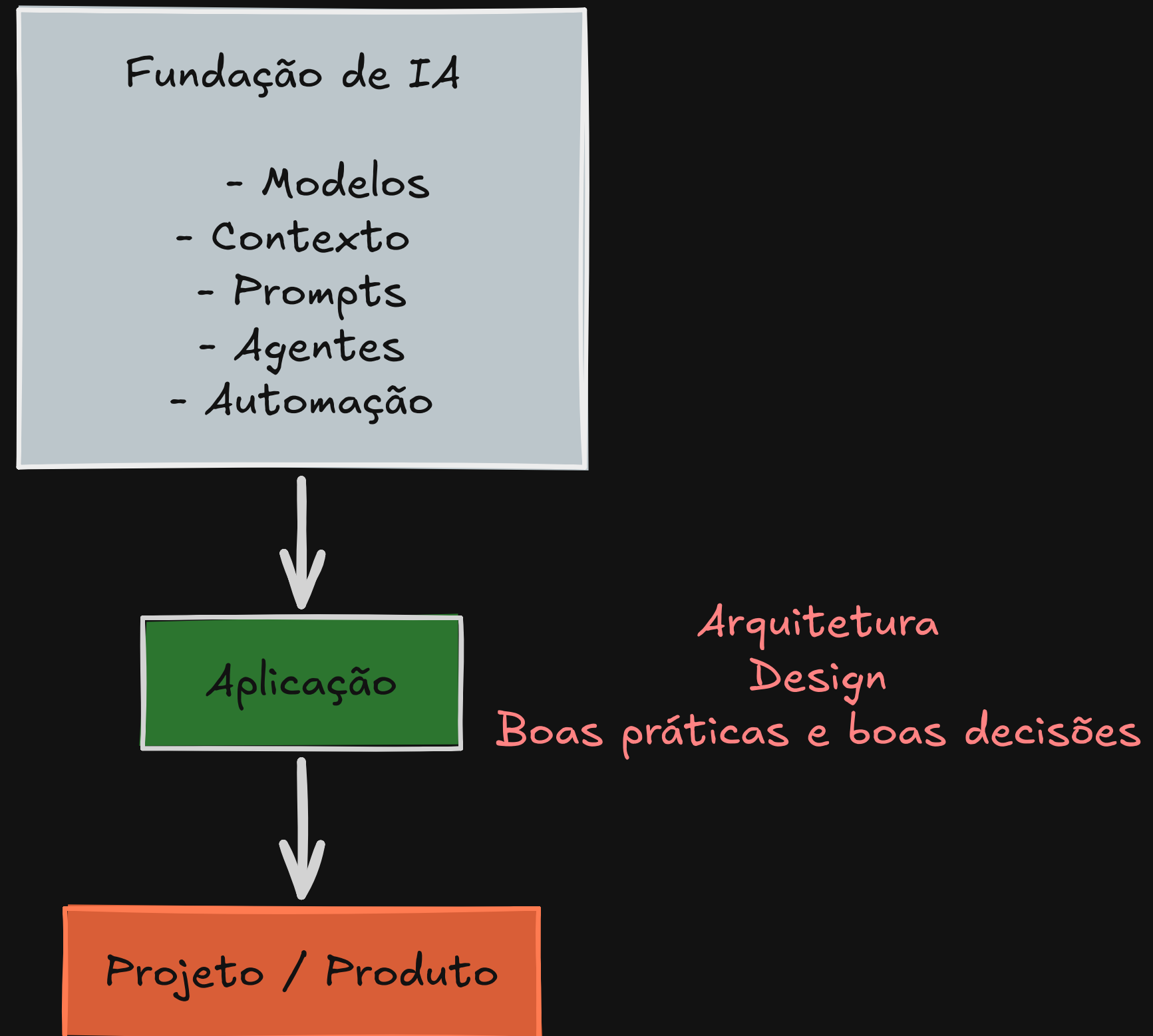
Dinâmica de desenvolvimento do projeto

- Back-end (Nest.js)
- Fundação de IA para Back-end
- Front-end (Next.js)
- Fundação de IA para Front-end
- Dev Fullstack - ambos ao mesmo tempo

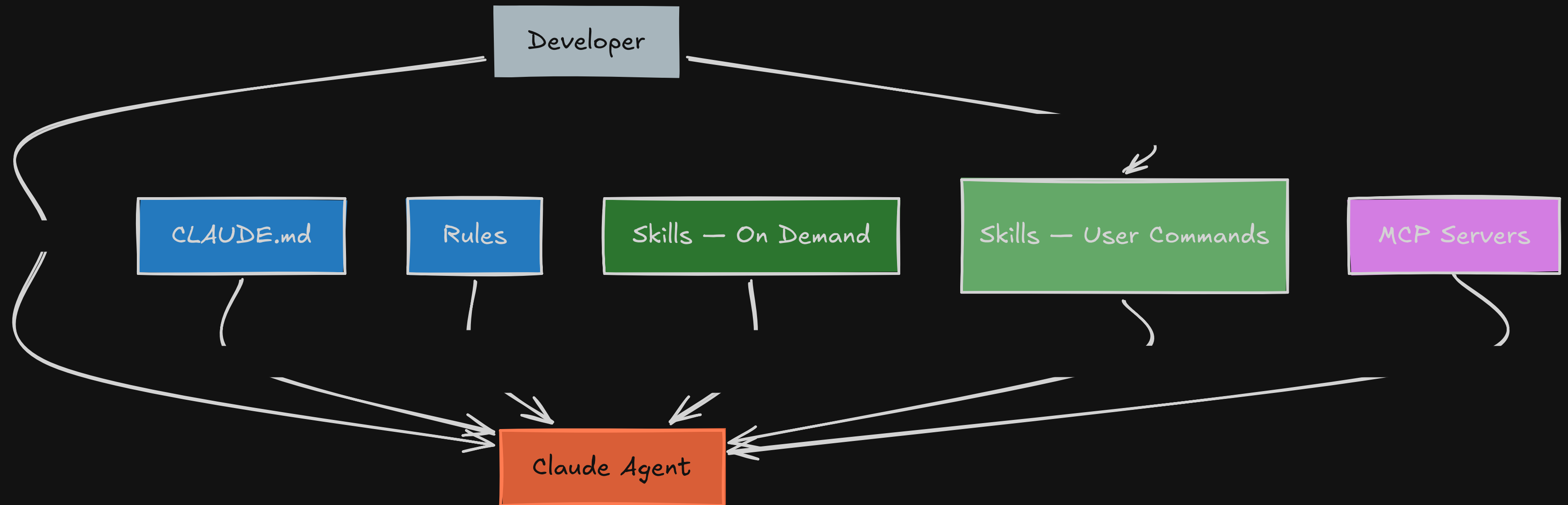
Workflows de desenvolvimento com IA



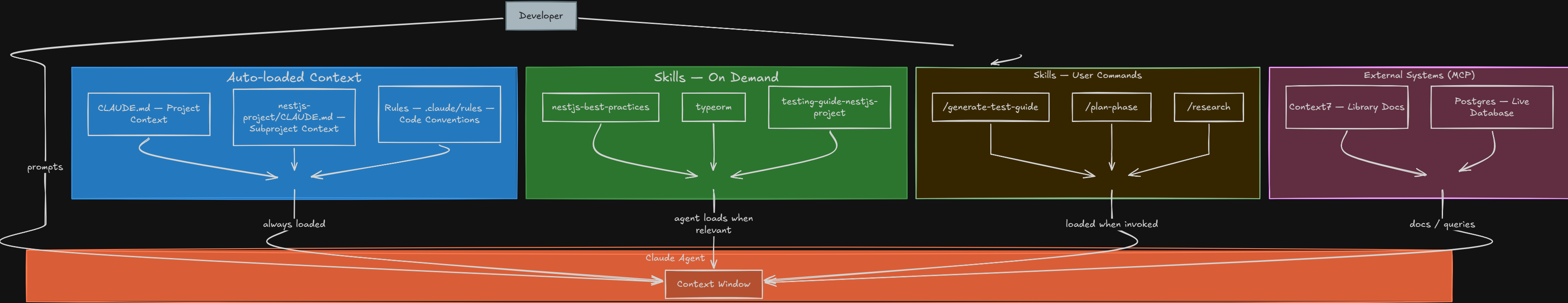
Fundação da IA



Artefatos de fundação com Claude Code



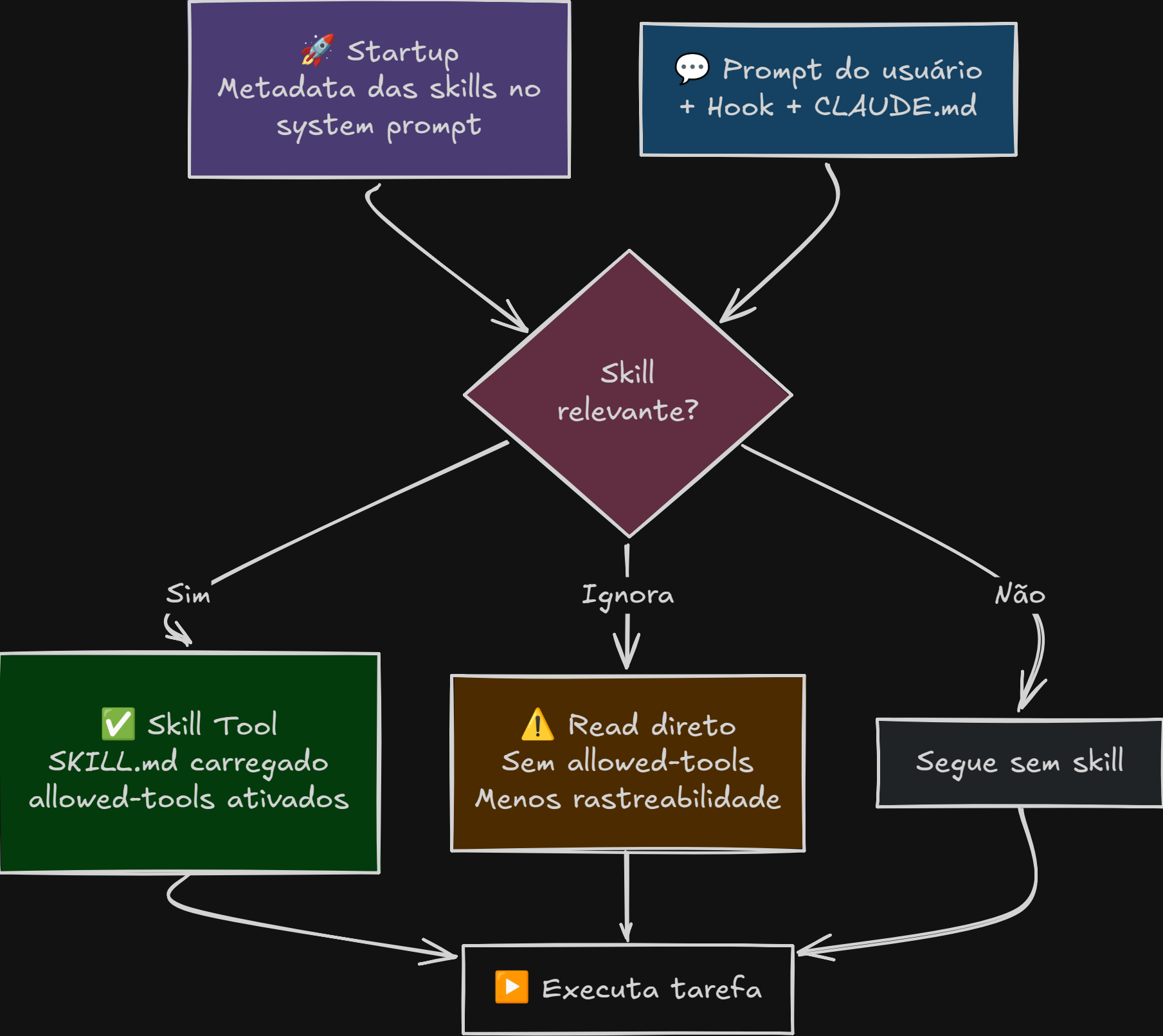
Fundação de IA do nosso projeto



Avaliando a própria SKILL e de terceiros

- Li o SKILL.md e todos os arquivos auxiliares por completo?
- A description é precisa? Não vai disparar falsos positivos/negativos?
- A estrutura é modular (index leve + referências lazy)? (Para skill grandes)
- O impacto na context window é aceitável?
- Não há instruções suspeitas, prompt injection ou chamadas externas estranhas?
- Testei em cenários reais do meu projeto?
- Testei em mais de um modelo (se aplicável)?
- A skill é compatível com meu stack e convenções?
- Não conflita com outras skills ativas?
- Sei quem é o autor e se a skill é mantida?

Fluxo de ativação de skills



4 ativação de skills é incerta.

Modelos menores pode não inferir que devem ativar.

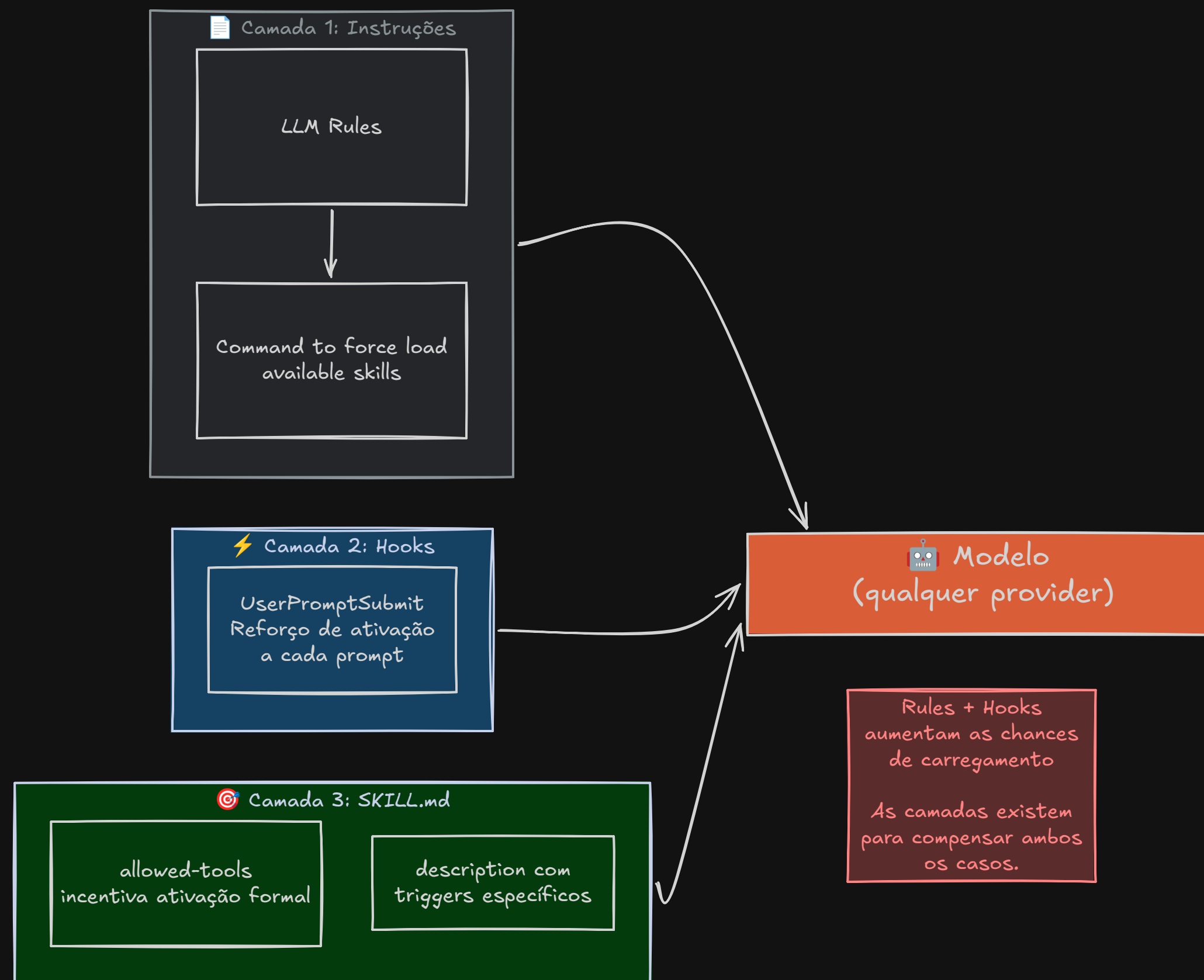
Modelos maiores acham que não precisam.

treinamento

nestjs-best-pp

typeorm

Arquitetura de Reforço de Ativação de Skills



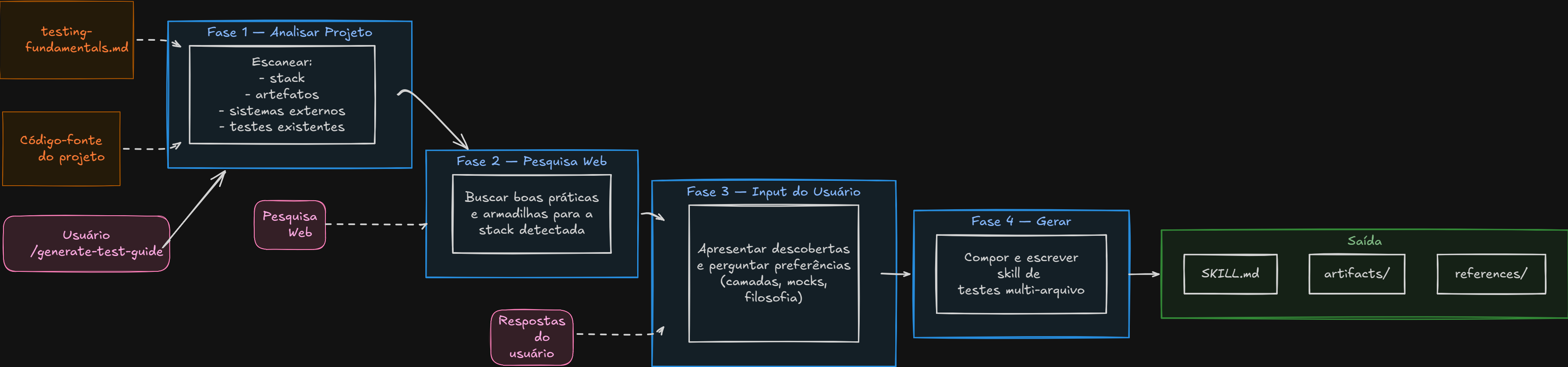
Skill de guia de testes do projeto

Porque testes são importantes para a IA?

- Testes são o feedback loop da IA
 - sem eles, a IA gera código "no escuro" sem saber se está correto. Com eles, roda, vê o que falhou, corrige e itera sozinha
- Definem comportamento esperado de forma executável
 - asserções concretas são mais precisas que descrições em linguagem natural como spec pro agente
- Testes ruins dão sinal errado
 - testes espelho ou de wiring fazem a IA otimizar pra passar no teste sem entregar comportamento correto. Pior que não ter teste
- Habilitam refactoring seguro
 - a IA é boa em refatorar, mas só com confiança se existe rede de testes validando que o comportamento não quebrou
- São o guardrail do modo autônomo
 - ferramentas agênticas como Claude Code dependem de rodar testes pra validar suas próprias mudanças. Melhor suíte = mais autonomia com segurança

Resumo: testes bem feitos transformam a IA de gerador de código em agente que verifica, corrige e evolui código com confiança

Diagrama da skill de guia de testes



Fundamentos de testes

****O que testar:****

- Lógica de negócio com branches, state machines, permissões
- Segurança (auth, rate limiting, sanitização)
- Integridade de dados (transformações, cálculos, migrações)
- Tratamento de erros e falhas de serviços externos
- Fluxos críticos do usuário (auth, pagamento, CRUD core)
- Race conditions e edge cases
- Contratos de fronteira (queries, mapeamentos, chamadas HTTP)
- Configuração de dependências (JWT, cache, throttle) — com instâncias reais, não mocks
- Configuração de módulos/DI — testar que compila e resolve dependências

****O que NÃO testar:****

- Comportamento garantido pelo framework
- Validação passthrough (1 teste por endpoint basta)
- Testes espelho que copiam o retorno da implementação
- Cobertura duplicada entre camadas
- Testes de wiring puro
- Estrutura estática (tipos, campos, valores iniciais)
- Variações de input no mesmo code path

****Mocks:****

- Só em fronteiras arquiteturais, nunca entre colaboradores internos
- Se precisa de muitos mocks → reescreva como integration test ou quebre a unidade

****Camadas:****

- ****Unit**** → lógica correta? Branches, cálculos, regras de domínio
- ****Integration**** → contrato com sistema externo correto? DB, fila, cache, HTTP client
- ****E2E**** → contrato HTTP correto? Status codes, validação, access control, response format

****Real vs Fake:****

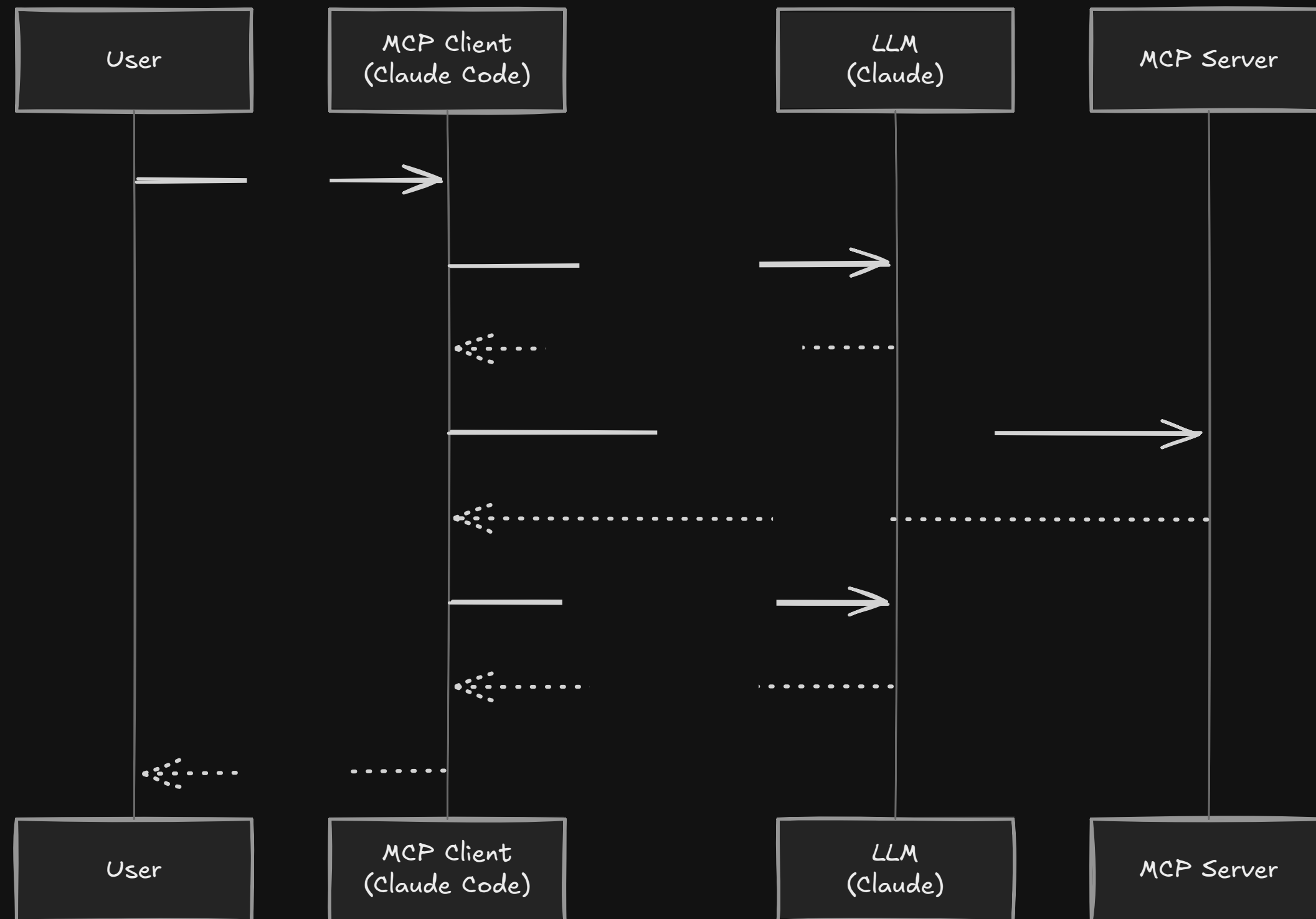
- Real (Docker): DB, fila, Redis — roda local, rápido, sem custo
- Fake: email, APIs externas, pagamento, file storage — custo, flakiness ou side-effects

****Princípio central:****

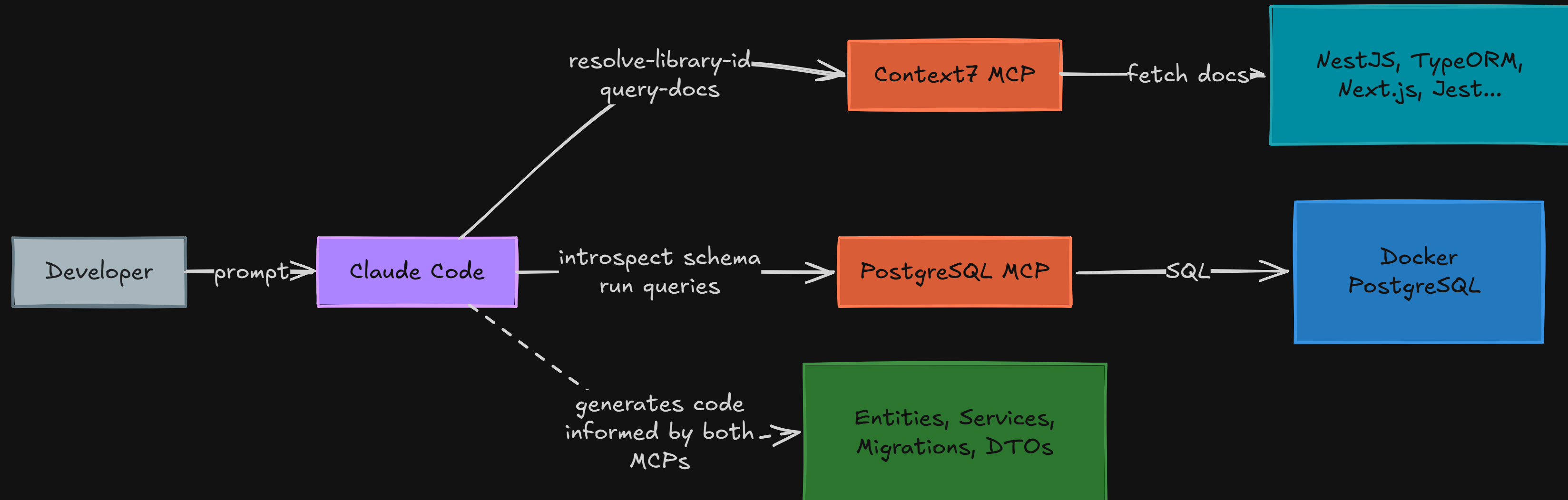
- Unit test que mocka dependência B não testa B — B precisa dos seus próprios integration tests
- Ambas as camadas são necessárias, nenhuma substitui a outra

MCP (Model Context Protocol)

Diagrama de funcionamento do MCP



Workflow do MCP no projeto



Workflow de desenvolvimento

Diagrama do Workflow de desenvolvimento baseado em RPI (Research-Plan-Implent)



Diagrama do planejamento de implementação da fase

fase 02 - auth

- access token
- refresh token (stateless vs stateful)

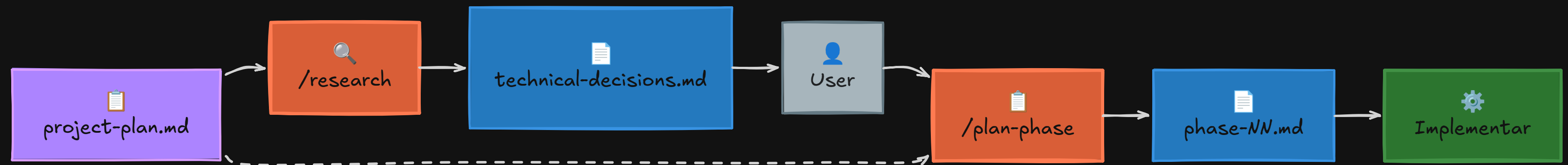


Diagrama da skill de pesquisa e tomada de decisões técnicas

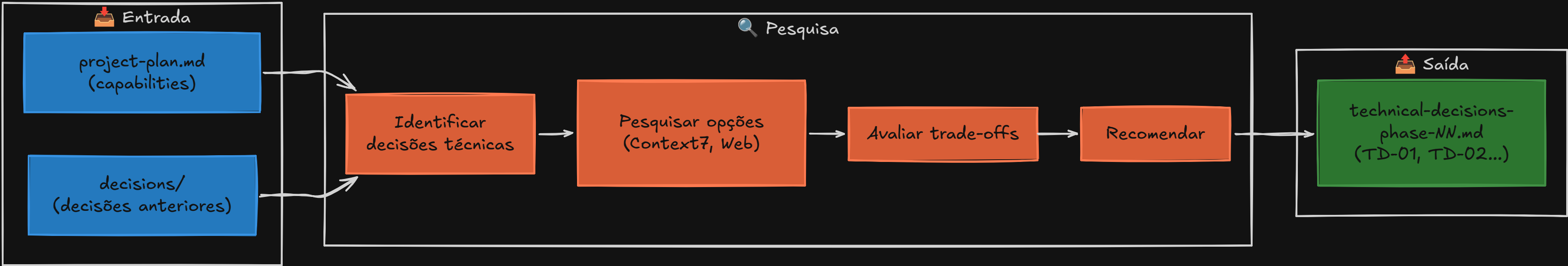


Diagrama do planejamento de implementação da fase

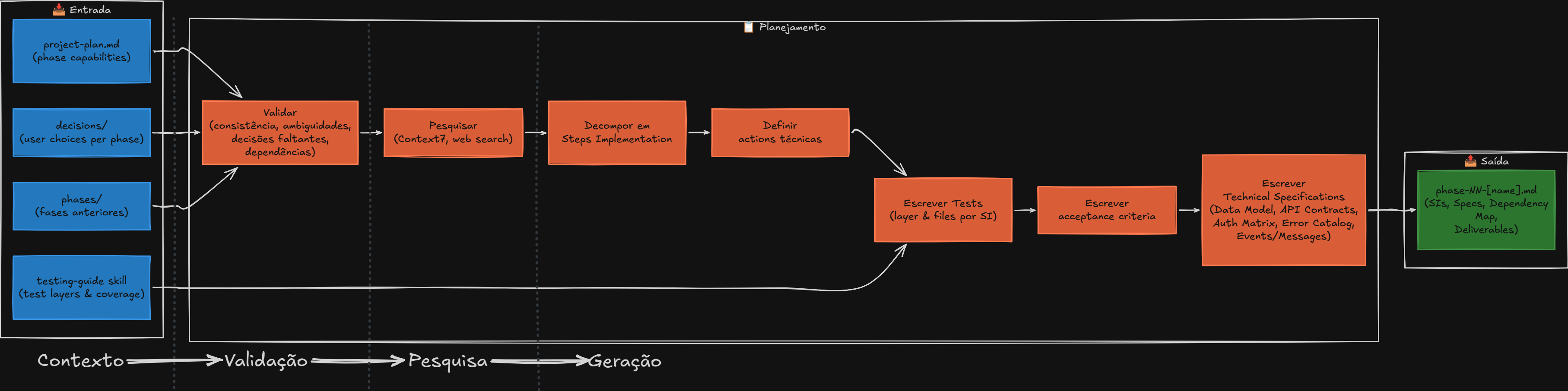
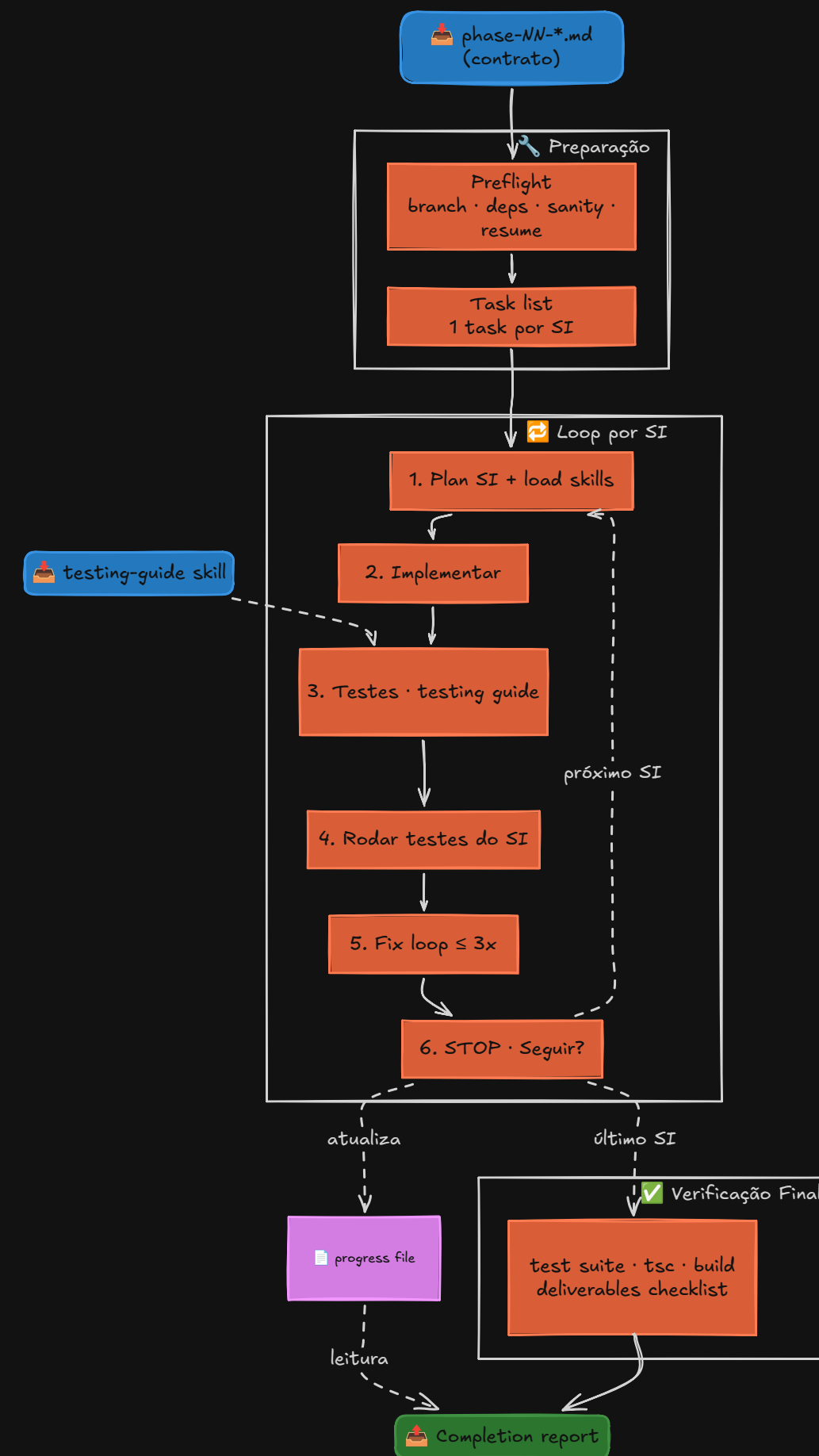
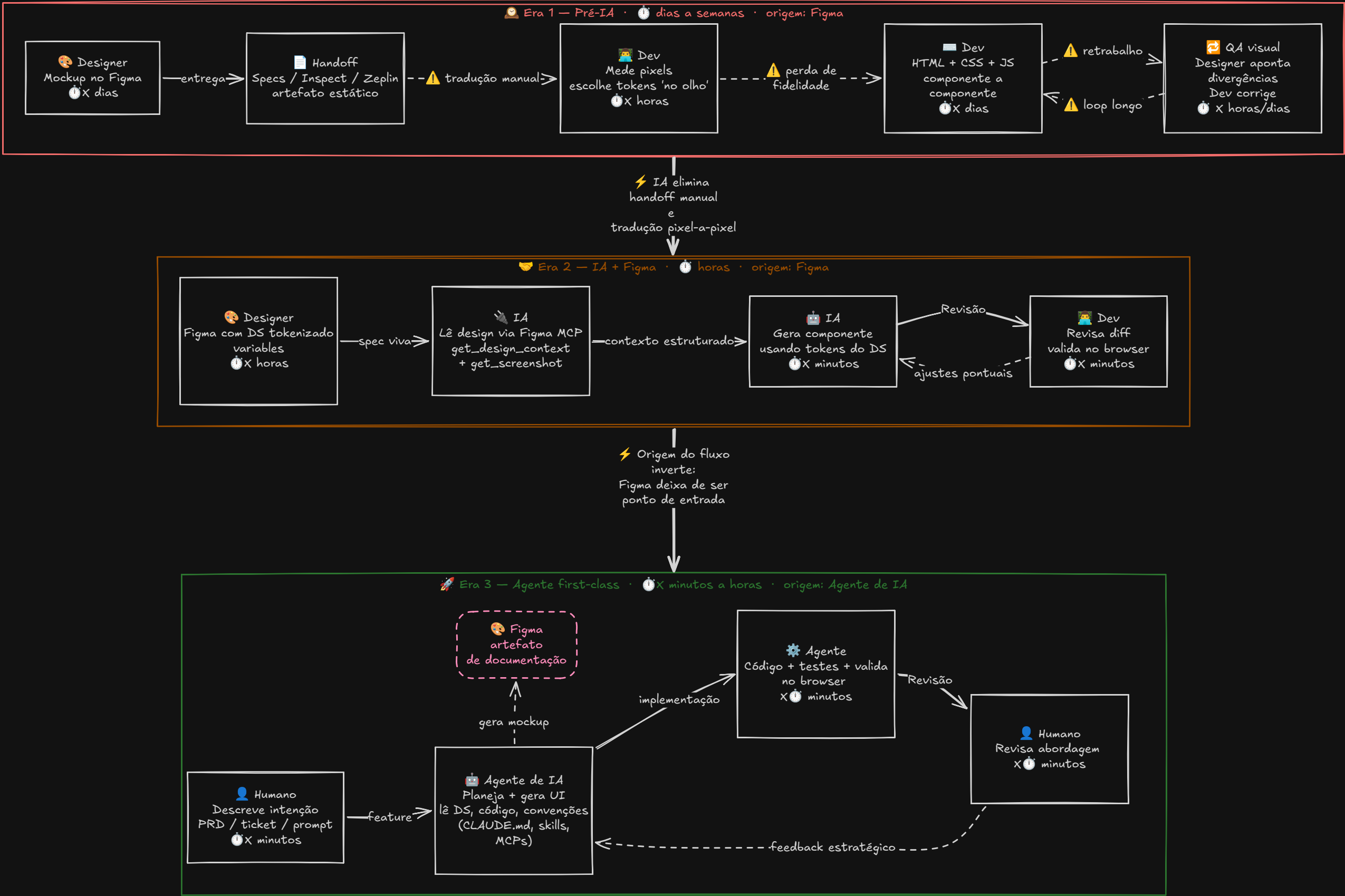


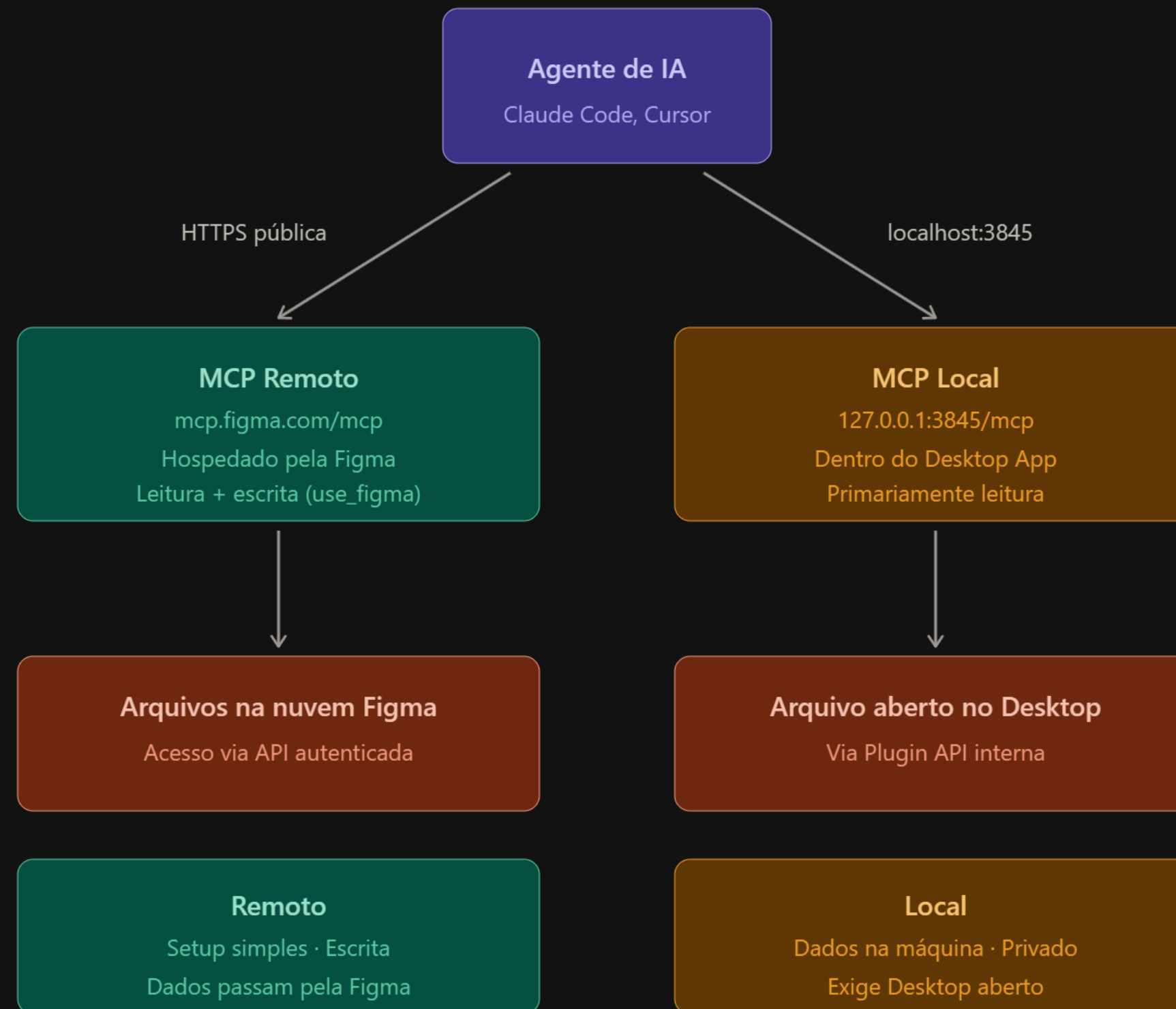
Diagrama da implementação da fase



Front-end na era de IA



Figma MCP



Ferramentas para criar design que se integram com IA

Ferramentas tradicionais (já existiam sem IA)

- Figma
- Sketch

Ferramentas AI native (IA é o centro da ferramenta)

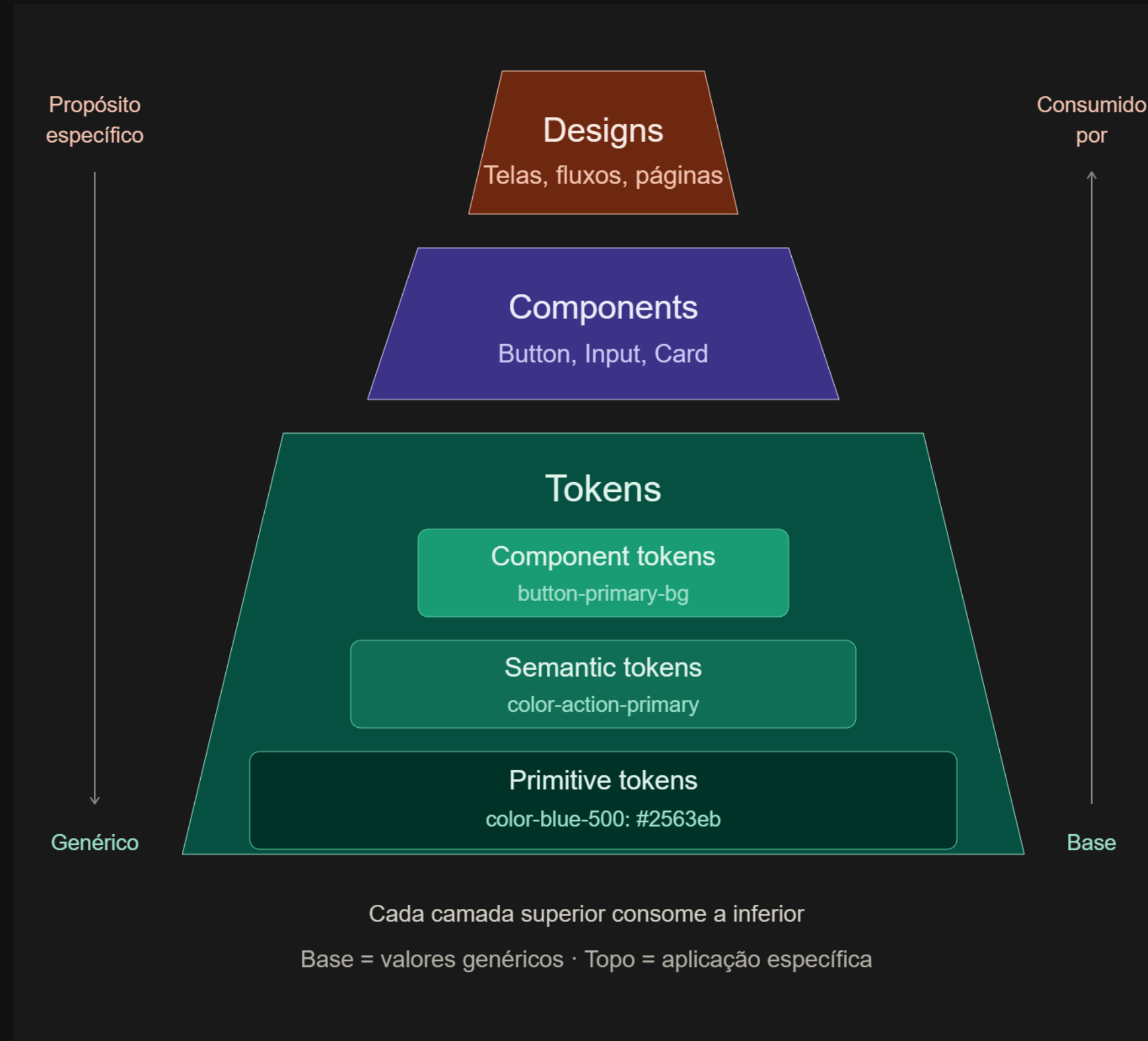
- Paper design
- Pencil dev
- Google Stitch
- Claude Design
- Magic Patterns

Design tokens

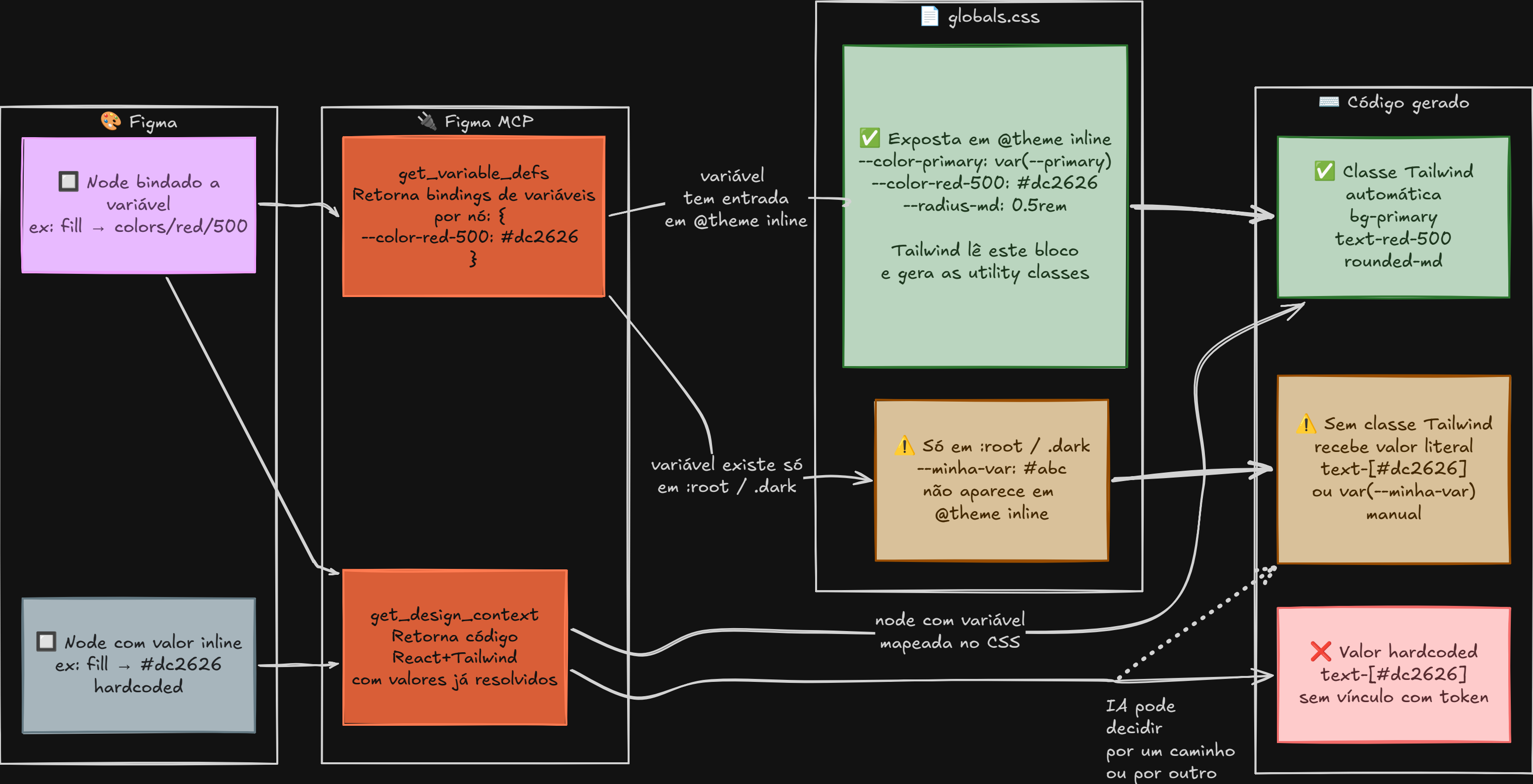
Design tokens são variáveis nomeadas que armazenam decisões de design. Em vez de espalhar valores brutos (cores em hex, tamanhos em pixels, durações em ms) por todo lugar, você define uma vez e referencia pelo nome.

```
/* Sem tokens - valores espalhados */  
.button-primary { background: #2563eb; padding: 12px 24px; }  
.link { color: #2563eb; }  
.badge-info { border-color: #2563eb; }
```

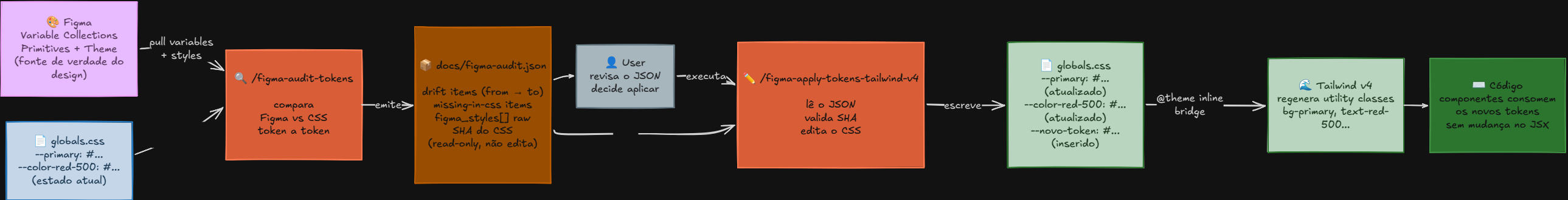
```
/* Com tokens */  
:root {  
  --color-primary: #2563eb;  
  --spacing-md: 12px;  
  --spacing-lg: 24px;  
}  
  
.button-primary {  
  background: var(--color-primary);  
  padding: var(--spacing-md) var(--spacing-lg);  
}  
  
.link {  
  color: var(--color-primary);  
}  
  
.badge-info {  
  border-color: var(--color-primary);  
}
```

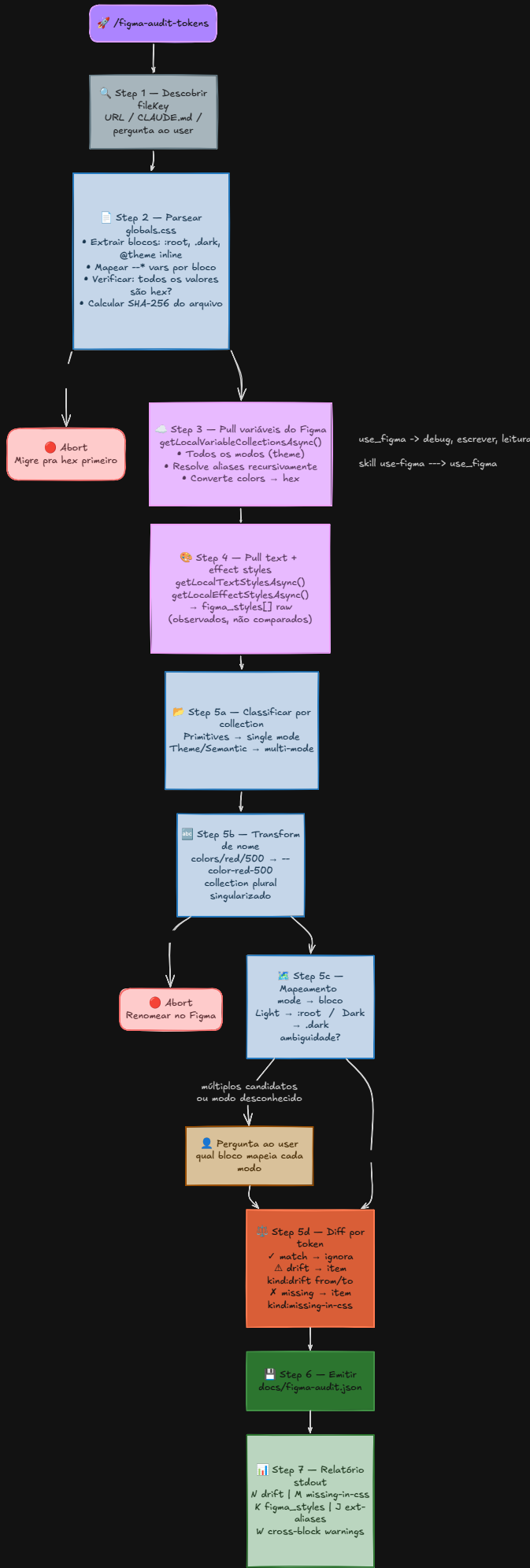


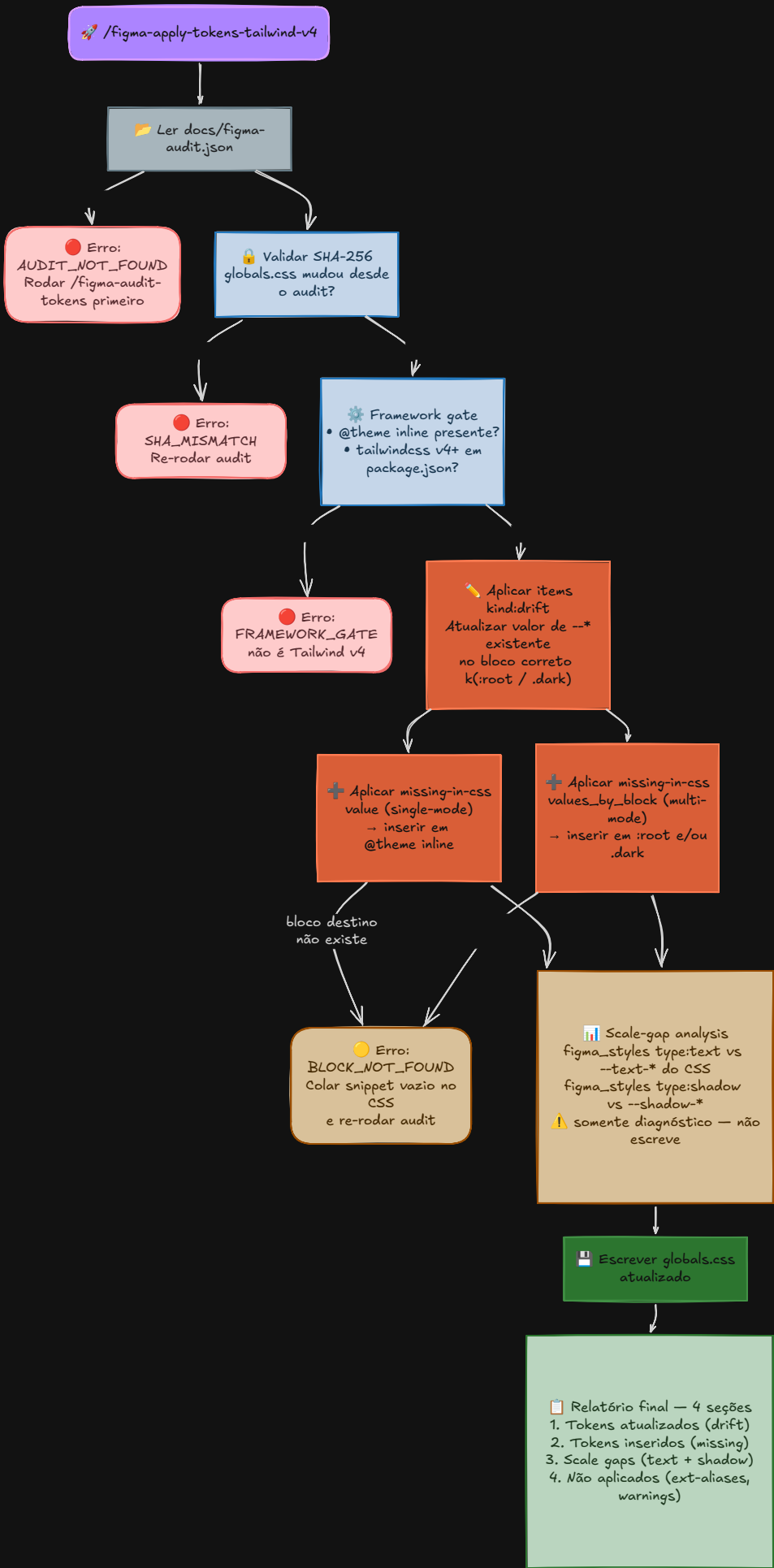
Tradução Figma <=> Código



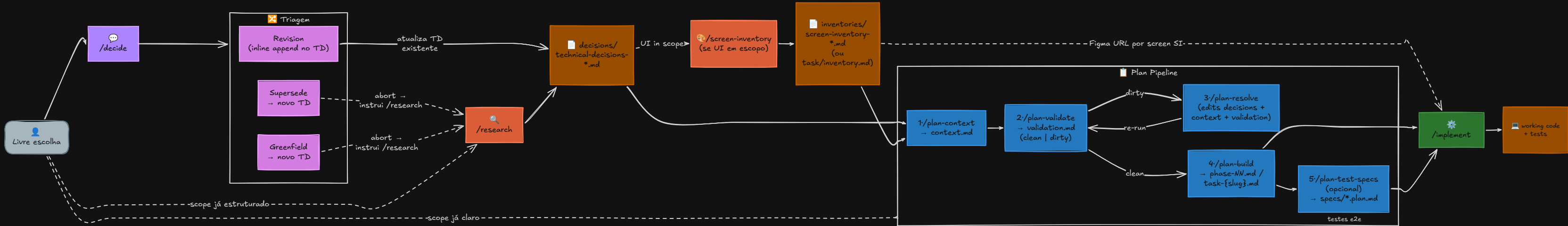
Sincronização de tokens do Figma com o projeto







Workflow de desenvolvimento orientado a etapas



ESTRUTURA DE ARQUIVOS GERADA

Phase mode (ex.: phase-02-auth-frontend, NN=02, slug=auth-frontend):

- docs/decisions/technical-decisions-{slug}.md -> /research
 - scope_type: phase
 - related_phases: [NN]
- docs/phases/phase-NN-{slug}/
 - context.md -> /plan-context
 - validation.md -> /plan-validate (clean|dirty)
 - library-refs.md -> /plan-resolve (cache Context?)
 - phase-NN-{slug}.md -> /plan-build (plan Final)
 - progress.md -> /implement
- {subproject}/specs/{scenario}.plan.md -> /plan-test-specs (opcional)

Task mode (ex.: task-msw-auth-handlers, slug=msw-auth-handlers):

- docs/decisions/technical-decisions-{slug}.md -> /research opcional
 - scope_type: ad-hoc
 - related_phases: [] ou [NN]
- docs/tasks/task-{slug}/
 - mesmos 5 arquivos (context.md, validation.md, library-refs.md, task-{slug}.md, progress.md) com as mesmas skills autoras
- {subproject}/specs/*.plan.md -> /plan-test-specs (opcional)

Workflow: /decide → /research → /screen-inventory → Plan Pipeline → /implement

 /decide — triagem free-text

- Revision: mesma Option letter → append inline no TD
- Supersede: Option letter muda → abort /research
- Greenfield: nenhum TD sobre tema → abort /research

Usa decisions-correlator. Não decide — só roteia.

 /research — explora opções, escreve TDs

- Produz technical-decisions-{slug}.md (status: pending)
- 2–4 Options/TD + Pros · Cons · Recommendation
- Capability gate: cada TD → bullet do project-plan
- Manifests + lockfiles + Context7 + web search

Não decide — user preenche **Decision:** depois.

 /screen-inventory (só se UI em escopo)

- Sub-agents paralelos: 1 por tela → Figma MCP
- Classifica: Presentational | Local-interactive | Server-connected
- Verbs of intent (infinitivo) → capability verbatim
- Reuse?/In DS? via filesystem snapshot (NÃO Code Connect)
- Marca (new) → sinal de bootstrap p/ plan-build

 1. /plan-context — consolidator read-only

6 subagents em paralelo:

 plan-reader (só phase)

 decisions-correlator

 decisions-reader

 decisions-detail-reader

 phases-reader

 inventory-digest-reader

Output: context.md — única fonte que stages downstream leem

 2. /plan-validate

Lê SÓ context.md
(zero outros file reads)

Issues:
MD-N · IC-N · UIG-N · OQ-N

Verdict: clean | dirty

3. /plan-resolve (se dirty)

- AskUserQuestion em batch
- Edita decisions + context + validation
- Context7 p/ libs decididas → library-refs.md

Depois: /plan-validate de novo

 4. /plan-build — escreve o plano final

- Step Implementations (SIs) + Technical Specifications
- Filtra subsections por Scope dos TDs (BE/FE/Cross/Repo)
- UI: SI Xa (Figma) + SI Xb (wiring + tests)
- Default append-mode (--rebuild força fresh)

Output: phase-MN-{slug}.md ou task-{slug}.md + progress.md

 5. /plan-test-specs (opcional)

Só dispara se SIs do plano carregam **Test Specs:**

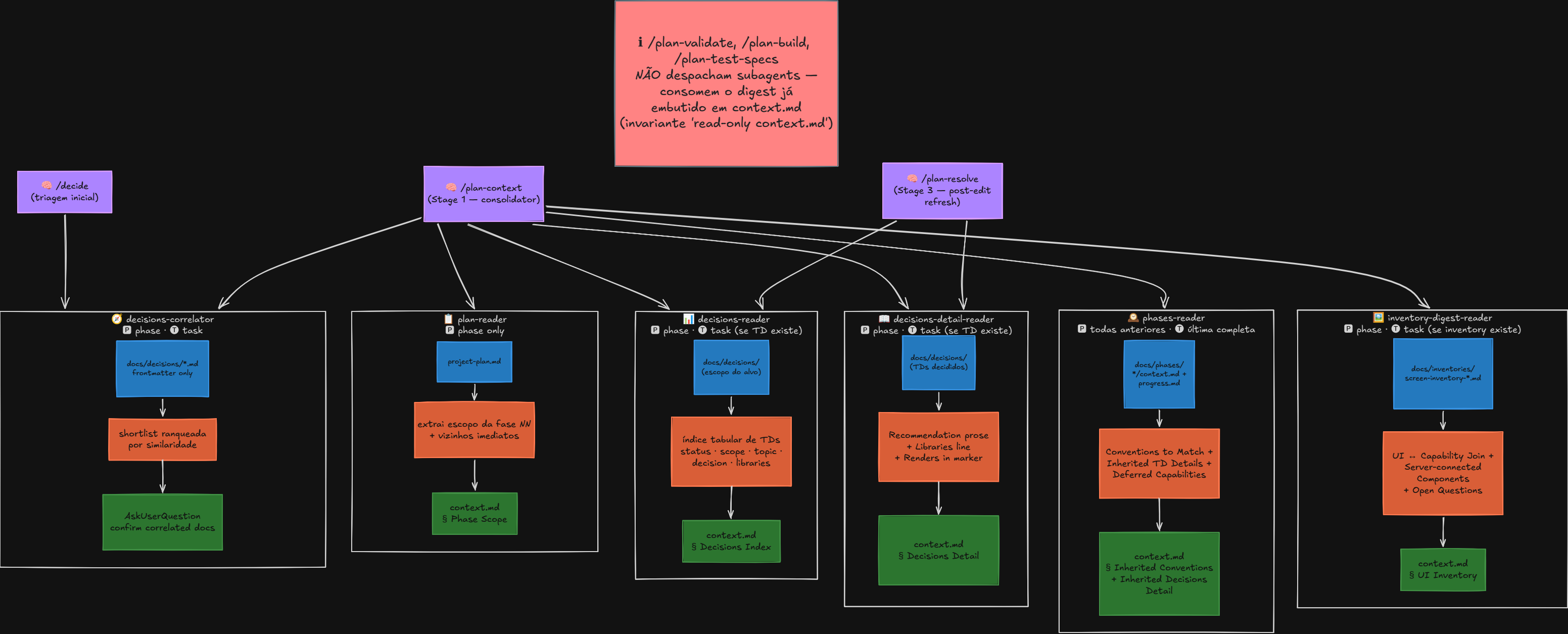
Output: <subproject>/specs/<scenario>.plan.md

 /implement — execução SI por SI

- Lê o plano como contrato (não decide nada)
- UM SI por vez: implementa → roda tests → só avança se passar
- Explore subagent p/ conventions da subproject
- general-purpose subagent p/ rodar tests sem poluir contexto
- UI: figma-implement-design com Figma URL do inventory

Output:  código + testes funcionando

Subagents personalizados do workflow de desenvolvimento



Subagents — padrão de compressão de contexto

Os 6 subagents em `.claude/agents/` existem por UM motivo: proteger a janela de contexto da thread principal. A mermaid mostra o QUE cada um faz. Este painel mostra o *GANHO* concreto + o modelo de execução.

❌ Sem subagents — main thread lê tudo

FONTES no disco que `/plan-context` precisa consultar pra montar a phase *NN*:

	project-plan.md	~5 KB
	8 × decisions/*.md	~30 KB
	3 × phases/*/context.md	~40 KB
	3 × phases/*/progress.md	~10 KB
	1 × inventories/screen-*.md	~20 KB

TOTAL raw na main thread ~105 KB

Custo:

- ~30% do context window queimado antes de raciocinar uma linha
- Cada re-leitura repete o custo
- Stages downstream (validate, build) reabrem tudo de novo, multiplicando o desperdício

→ Solução ingênua não escala além de phases pequenas.

✅ Com subagents — compressão em paralelo

Cada subagent roda em **CONTEXT WINDOW PRÓPRIO**, lê só seu slice das fontes, retorna um digest:

raw ~5 KB	→	plan-reader	≈ 200 B
raw ~30 KB	→	decisions-reader	≈ 1 KB
raw ~30 KB	→	decisions-detail	≈ 2 KB
raw ~30 KB	→	decisions-correlator	≈ 300 B
raw ~50 KB	→	phases-reader	≈ 2 KB
raw ~20 KB	→	inventory-digest	≈ 1.5 KB

raw ~105 KB → digest TOTAL ~7 KB

Main thread só vê os 7 KB consolidados. Despachadas as 6 chamadas em *UMA* mensagem (Agent tool × 6 em paralelo), retornam no mesmo turn.

Ganho real:

- ~93% de redução de payload
- Main thread reserva o context window pra raciocínio + escrita do plan
- Stages downstream consomem o mesmo digest sem custo adicional

🔄 Modelo de execução — um turn do `/plan-context`

t₀ → User invoca `/plan-context NN`

t₁ → Main thread: resolve mode + slug, bounded-read da fase no `project-plan.md`, compõe `scope_prose`.
Custo: ~500 B no contexto principal.

t₂ → Main thread emite *UMA* mensagem com 5-6 chamadas Agent paralelas:

plan-reader		decisions-reader		decisions-detail-reader		decisions-correlator		phases-reader		[inventory-digest-reader]
-------------	--	------------------	--	-------------------------	--	----------------------	--	---------------	--	---------------------------

Cada uma roda num **CONTEXT WINDOW ISOLADO**. Read-only. Não se enxergam. Não escrevem nada.

t₃ → Todas retornam no mesmo turn. Main thread recebe 6 blocos markdown compactos (~7 KB total).

t₄ → Main thread monta `context.md` (Phase Scope + Decisions Index + Decisions Detail + Inherited Conventions + UI Inventory) e grava.

🧩 Invariantes que tornam o padrão seguro

 Read-only

Nenhum subagent escreve em disco.

Fontes (decisions/, phases/, inventories/, `project-plan.md`) são imutáveis sob a perspectiva do agent.

Mutações vivem em `/plan-resolve` (via Edit/Write no main thread).

 Isolamento de contexto

Cada Agent call tem seu próprio window.

Subagents *NÃO* veem a conversa principal, *NÃO* veem outros subagents, *NÃO* acumulam state entre invocações.

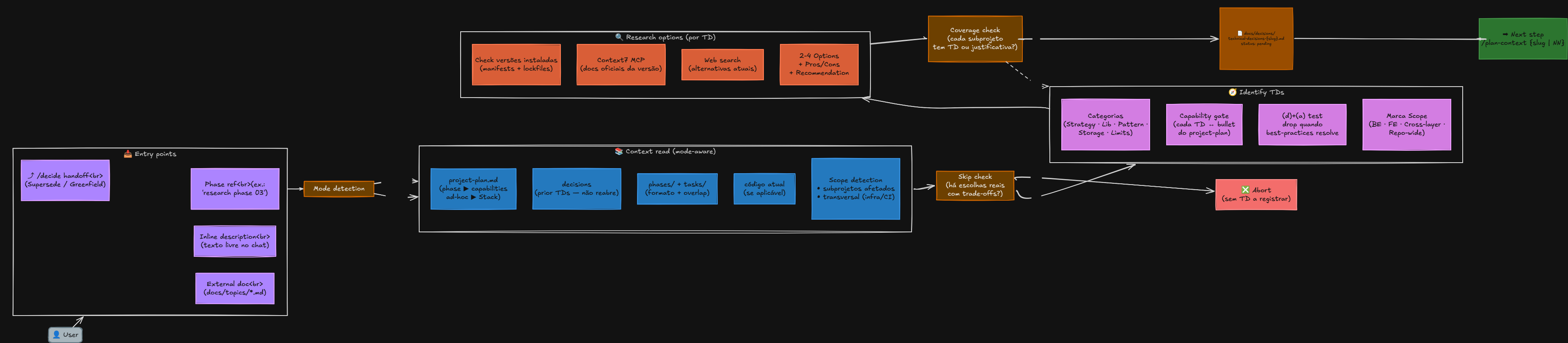
O contrato é o prompt + o que retornam. Reproduzível por construção.

 Re-dispatch sob demanda

`/plan-resolve` re-roda `decisions-reader` + `decisions-detail-reader` pós-edit, e reescreve as 2 seções correspondentes no `context.md`.

O digest fica sempre coerente com o estado atual do disco — sem o main thread reler os arquivos crus.

Skill de Research aprimorada



Anatomia de um TD — o que o /research escreve no disco

docs/decisions/technical-decisions-{slug}.md

```
--- frontmatter ---
scope_type: phase | ad-hoc
related_phases: [NN] | [] (integer array; [] = puro global)
status: pending → decided (flipado por /plan-resolve)
date: YYYY-MM-DD
scope_description: "linha curta"
covers_capabilities: [...] (opcional, define slice de phase)
depends_on_slices: [sibling-slug, ...] (opcional, DAG inter-slice)

# Technical Decisions - {titulo}

## TD-01: [nome da decisao] (ref externa: {slug}/TD-NN)

**Scope:** Backend | Frontend | Cross-layer | Repo-wide
→ /plan-build filtra Tech Specs subsections por esse campo

**Capability:** "<bullet literal do project-plan.md>" (phase mode)
ou
**Trigger:** "<one-liner da pergunta do user>" (ad-hoc, related_phases: [])

**Context:** Por que decidir; qual scope element depende.

**Options:** 2 a 4 opcoes. Menos que 2 nao e decisao. Mais que 4 e ruído.

### Option A: [Nome curto]
- Como funciona (2-3 frases)
- Pros: vantagens concretas no stack
- Cons: desvantagens concretas

### Option B: [Nome] ...idem
### Option C: [Nome] ...idem (opcional)

**Recommendation:** Option X - frase justificando no contexto.
→ /research escreve; user pode mudar antes de /plan-resolve

**Decision:** _[pending]_ → preenchido por /plan-resolve ou /decide (Revision)

**Renders in:** ui-contracts | frontend-runtime (opcional, so Frontend)

**Libraries:** argon2, @nestjs/jwt → preenchido por /plan-resolve (Context? auto)

**Revisions:** append-only, criado por /decide na branch Revision
- YYYY-MM-DD - descricao. Rationale: motivo.
```

Teste (d)+(a) — vira TD ou descarta?

```
Antes de escrever qualquer TD, aplique:

(d) Best-practices SKILL ja resolve isso?
→ typeorm, nestjs-best-practices,
next-best-practices,
vercel-react-best-practices,
ou framework docs via Context?

(a) Item e citado em ≥2 arquivos que
precisam ficar consistentes?
→ ex: Joi schema + compose.yaml + .env
OU contrato BE ↔ FE
OU shared types FE ↔ BE

REGRA (binaria):
(d) SIM AND (a) NAO → DROP
senao → KEEP

--- DROP (implementacao) -----

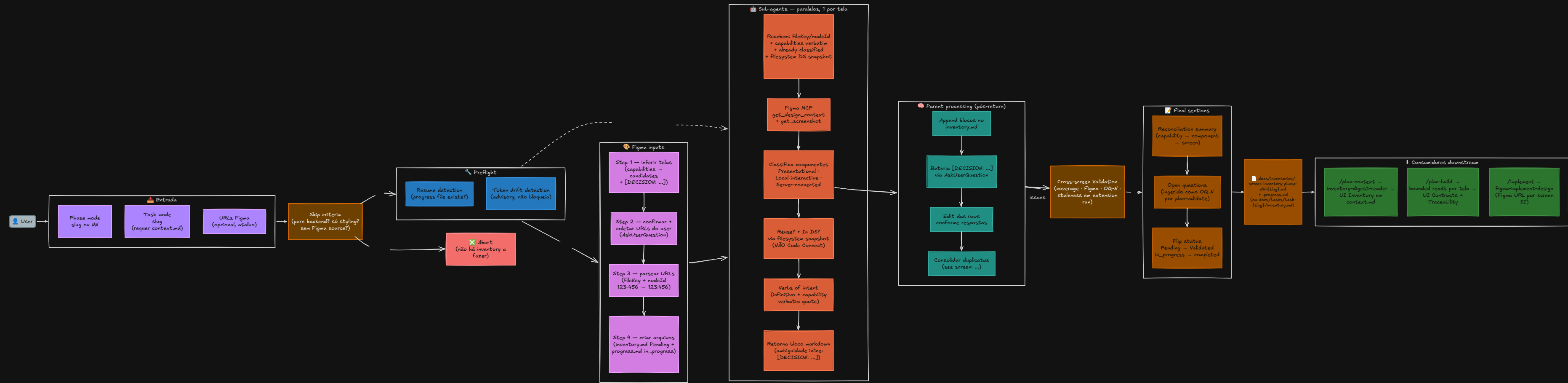
- Export form (default vs named)
- Glob patterns (*.entity.ts)
- Directory naming (src/database/seeds/)
- npm script names (migration:generate)
- Framework defaults (ConfigModule.isGlobal,
TypeOrmModule.synchronize)
- Container healthcheck idioms

--- KEEP (estrategico) -----

- Token storage (cookie vs header)
→ cross-component contract BE↔FE
- Env var keys canonical set
→ schema + compose + runtime alinhados
- Auth strategy (stateless vs stateful)
→ nenhuma skill resolve sozinha
- Upload protocol (tus vs multipart)
→ cross-component + viables genuinos
- Pagination (cursor vs offset vs page)
→ cross-layer (BE query + FE UI)
- Hashing algo (bcrypt vs argon2)
→ non-functional + lockfile constraint
```

Phase mode vs Ad-hoc mode

	PHASE ----	AD-HOC -----
Trigger	/research NN /research {slug}	/research "texto" ou via /decide
Scope source	project-plan.md capabilities da fase NN	user description + external doc
Frontmatter	scope_type: phase related_phases: [NN]	scope_type: ad-hoc related_phases: [NN+] ou []
Per-TD field	**Capability:** (bullet verbatim)	se [NN+]: **Capability:** (bullet verbatim) se []: **Trigger:** (one-liner)
Capability Gate	ON (TD ↔ bullet)	ON se [NN+] OFF se []
Slicing	covers_caps depends_on_slices	N/A
Filename	technical- {phase-slug}.md	technical- {topic-slug}.md
Handoff	/plan-context NN	/plan-context {slug} (bootstrappa docs/tasks/ task-{slug}/)



Anatomia do screen-inventory.md — o que a skill escreve no disco

docs/inventories/screen-inventory-phase-NN-{slug}.md + .progress.md (ou docs/tasks/task-{slug}/inventory.md)

```
--- frontmatter ---
scope_type:      phase | task
related_phases:  [NN]                                (integer array)
status:          Pending → Validated                  (flipado pela validation step)
date:            YYYY-MM-DD
figma_file_key:  {fileKey}                            (uma file_key por inventory)

# Screen Inventory - Phase NN

## Screen: {Nome da tela}                                (um bloco ## por tela)

**Route:**      /caminho/{:param}
**Figma URL:**   https://figma.com/design/{fileKey}/...?node-id=N:M
**Capabilities:**
  - "<bullet literal do project-plan.md"
  - "<...>"

### Verbs of intent                                       (intent-level, NAO impl-level)

| Verb (infinitivo) | Capability quote | Component | Layer |
|-----|-----|-----|-----|
| Submeter login    | "users can sign in" | LoginForm | Server-connected |
| Validar campos     | "users can sign in" | LoginForm | Local-interactive |
| Exibir erro        | "users can sign in" | InlineAlert | Presentational |

### Components on screen
- LoginForm          (NEW · server-connected · in DS? no)
- InlineAlert        (REUSE · presentational · in DS? yes → components/ui/alert.tsx)
- Button             (REUSE · local-interactive · in DS? yes → components/ui/button.tsx)

[DECISION: rememberMe persiste via cookie ou localStorage?]
      ^^ marker inline injetado pelo sub-agent quando ele
      precisa do user. Batched via AskUserQuestion pelo
      parent, depois que todos sub-agents retornam.

## Reconciliation summary                                (secao final, parent gera)
capability → component(s) → screen(s)                    (uma row por capability)

## Open questions                                       (formato OQ-N)
OQ-1: <pergunta nao resolvida que sobrou> → ingerida por /plan-validate
OQ-2: <...>
```

3 layers — classifica componentes por comportamento

```
A categoria de cada componente decide-se
pelo SEU comportamento, nao pela aparencia.

-----
Presentational
-----
- Recebe props, renderiza markup.
- Zero state interno.
- Zero side effect.
- Trivial: input → markup.

e.g. InlineAlert, Badge, Avatar,
     VideoCardSkeleton, EmptyState

-----
Local-interactive
-----
- State interno (useState, refs).
- Disclosure, toggles, valid. local.
- Nao fala com servidor sozinho.
- Pode emitir eventos pra cima.

e.g. PasswordField (show/hide),
     Disclosure menu, Tabs,
     ClientForm (validacao local)

-----
Server-connected
-----
- Le ou escreve via fetch/action.
- Owns loading/error/success states.
- Tipicamente orquestrador da tela.
- Vira UI Contract no /plan-build.

e.g. LoginForm (POST /login),
     VideoUploadDialog,
     Commentlist, SubscribeButton

-----
REGRA pratica (decision tree)
-----
tem fetch/action? → server-connected
se nao tem state? → local-interactive
se nao             → presentational
```

Modelo de JOIN — capability <-> component <-> screen

```
A skill produz uma triade que precisa
fechar nos tres eixos:

  capability ↔ component ↔ screen

-----
[1] Coverage gate
-----
Toda capability UI-bearing do
project-plan.md aparece em ≥1 row.

Se faltar → /plan-validate emite
CV-N e /plan-resolve forca volta
pro /screen-inventory.

-----
[2] Server-connected → UI Contract
-----
Todo Server-connected component
vira candidato a UI Contract no
/plan-build (com Interactions,
Verbs of Intent, etc).

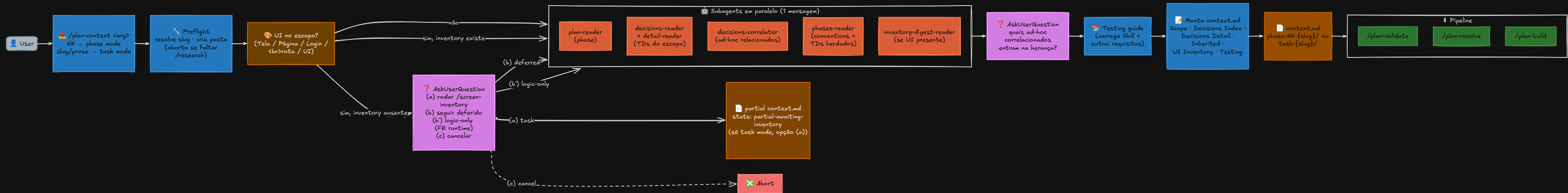
-----
[3] REUSE + in DS → path inline
-----
Componentes "REUSE + in DS" entram
com path pra components/ui/*.tsx
via FILESYSTEM SNAPSHOT.
(NAO Code Connect - decisao de design)

-----
Verb of intent — formato
-----
Infinitivo curto + quote literal da
capability. NAO e nome de funcao,
NAO e nome de componente.

ok:   "Submeter login"
ok:   "Validar campos do form"
ruim: "handleSubmit" (nome de fn)
ruim: "Login"         (sem verbo)

-----
[DECISION: ...] inline vs TD
-----
TD          [DECISION: ...]
--          -----
estrutural  tactical
/research   sub-agent injeta
Options letter pergunta livre
vira arquivo fica inline na row
→ /plan-build → AskUserQuestion
                batched pelo parent
```

Skill de planejamento de contexto



Anatomia do context.md — o que cada secao contem + quem a produz

docs/phases/phase-NN-{slug}/context.md (ou docs/tasks/task-{slug}/context.md - bootstrap automatico em task mode)

```
--- frontmatter ---
kind:      phase | task
name:      phase-NN-{slug} | task-{slug}
state:     [partial-awaiting-inventory] (efemero - so task mode, opcao (a) do Step 0.5; some na rerun)
sources_mtime: (upstream fingerprint - downstream aborta se ficar stale)
docs/project-plan.md: "ISO-8601" (phase only)
docs/decisions/technical-decisions-{slug}.md: "ISO-8601" (se existir)
...      1 linha por: ad-hoc related_phases NN, correlator-confirmed,
        prior-phase context.md, testing-guide skill
```

# {name} - Context	
## Scope	← plan-reader (phase) scope_prose verbatim (task) phase: Phase name + Capabilities + Out of scope + Deliverables + Affected subprojects + Deferred subprojects + Sequencing + Neighbors task: prose pura, sem sub-fields, sem capability gate
## Decisions Index	← decisions-reader tabela: Ref Source Scope Topic Status Decision Libraries + _Source files:_ subsection
## Capability Coverage	← derivada main-thread (PHASE MODE ONLY - omitida em task) tabela: Capability bullet (verbatim) Covered by ({slug}/TD-NN "-") "-": sera flagged como MD por /plan-validate
## Decisions Detail	← decisions-detail-reader (current scope so) ### {slug}/TD-NN **Recommendation:** prose (Option-X prefix stripped) **Libraries:** lib-a, lib-b
## Inherited Decisions Detail	← phases-reader + correlator-confirmed docs dedupe vs Decisions Detail (current_refs) inter-source: phases-reader vence correlator (vetted)
## Inherited Conventions	← phases-reader (flatten bullets _(from phase MM)_)
## Inherited Deferred Capabilities	← phases-reader (informational; plan-validate NAO fira)
## UI Inventory	← inventory-digest-reader placeholder direto (Step 7) + estados - vide painel da direita
## Non-UI / Deferred Capabilities	← placeholder _None._ (preenchido pelo /plan-resolve, append-only)
## Testing Requirements	← Skill tool: testing-guide={subproject} tabela: Artifact type Required layers (por subproject)

Matriz de subagents — quem roda em phase/task + saida

Subagent	Phase	Task	Saida → secao do context.md
plan-reader	X	-	## Scope (Phase name, Capabilities, Out of scope, Deliverables, Affected, Neighbors)
decisions-reader	X	X*	## Decisions Index (*task: so se TD doc existe; senao placeholder _No TDs...)
decisions-detail-reader	X	X*	## Decisions Detail (*task: so se TD doc existe)
decisions-correlator	X	X	## Inherited Decisions Detail pool: phase: ad-hoc related_phases:[] task: todas decisions except own
phases-reader	X	X	## Inherited Conventions ## Inherited Decisions Detail ## Inherited Deferred Capabilities phase: TODAS prior phases task: so latest completed phase
inventory-digest-reader	X†	X†	## UI Inventory (†dispatched APENAS se Step 0.5 detectou UI scope E inventory file existe; logic-only NUNCA dispatcha)
----- reads que o MAIN THREAD faz (nao-agente) -----			
- project-plan.md "### Fase NN"			(Step 0, phase scope_prose monolithic fallback)
- covers_capabilities frontmatter			(Step 0, phase slice)
- partial context.md ## Scope			(Step 0, task rerun-from-partial)
- Recommendation + Libraries dos			(Step 1, correlator-confirmed)
- glob docs/decisions/*.md			(Step 2, Filter Trace check)
- testing-guide SKILL.md			(Step 5)
Tudo o resto vem via subagent return.			

Os 4 estados do ## UI Inventory — token-anchors

Secao tem 4 estados mutuamente exclusivos. Plan-build Gate 9: plan-validate detectam via inferencia por token-anchor.	

[1] OMITTED - sem sinal UI no escopo	

## UI Inventory	NAO existe (nao emitido nem como placeholder)
Trigger:	nenhum match em Tela / Pagina / Area / Login / UI (case-insensitive substring)
Downstream:	ui_in_scope: false

[2] DEFERRED - opcao (b)	

## UI Inventory	
No screen inventory - UI↔API sync deferred. Run /screen-inventory ... and then rerun /plan-context <arg> to activate UI checks.	
Trigger:	ui_deferred: true
Token:	"_No screen inventory -"
Downstream:	ui_in_scope: false (UI checks desligados)

[3] LOGIC-ONLY - opcao (b')	

## UI Inventory	
Frontend-runtime only - no screen inventory needed for this phase. Run /screen-inventory <arg> if a UI surface is added in a future revision.	
Trigger:	ui_logic_only: true
Token:	"_Frontend-runtime only -"
Downstream:	ui_in_scope: logic-only (FE-runtime TDs OK; sem screens / UI Contracts)

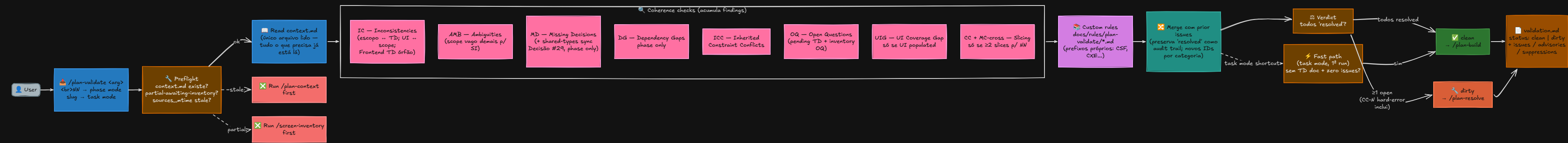
[4] POPULATED - inventory presente	

## UI Inventory	
Source: 'docs/inventories/...'`	
Screens in scope: N	
#### UI ↔ Capability Join (tabela)	
#### Server-connected Components (lista)	
#### Open Questions from Inventory (OQ-N)	
#### Inherited UI Components (so task)	
Downstream:	ui_in_scope: true + UI Contracts em /plan-build

Promocao LOGIC-ONLY → POPULATED	

1. /screen-inventory NN (gera arquivo)	
2. /plan-context NN (regen idempotente)	
→ flag flipa por Gate 9 inference (sem path novo de codigo)	

Skill de validação de contexto



Anatomia do validation.md — frontmatter como audit trail + body por categoria

docs/phases/phase-NN-{slug}/validation.md (Ou docs/tasks/task-{slug}/validation.md)

--- frontmatter --- kind: phase task name: phase-NN-{slug} task-{slug} status: clean dirty ('clean' iff TODO issue tem status: resolved) issue_count: N (open issues nesta revisao) sources_mtime: docs/(phases/tasks)/(dir)/context.md: "ISO-8601" (staleness check no preflight da proxima run) docs/decisions/technical-decisions-*: "ISO-8601" (1 por _Source files_: do context.md) docs/project-plan.md: "ISO-8601" (so se Check 8 rodou) issues: (LOAD-BEARING - audit trail preservado) - id: AMB-1 status: open (vira 'resolved' apos /plan-resolve) summary: "..." - id: MD-1 status: resolved (preservado em reruns - NUNCA renumerado/dropado) resolved_by: {slug}/TD-11 advisories: (phase + slicing only; NAO flipa status) - id: MC-cross-1 status: open summary: "..."	
# {name} - Validation	
## Findings	(open issues - categoria por sub-heading)
### Inconsistencies	IC-N (escopo ↔ TD; UI verb cita cap fora; Frontend TD orfa)
### Ambiguities	AMB-N (scope vago demais p/ derivar SIs)
### Missing Decisions	MD-N (cap sem TD + shared-types sync #29 phase only)
### Dependency Gaps	DG-N (phase only - never task)
### Inherited Constraint Conflicts	ICC-N (current TD contradiz inherited convention/TD)
### Unresolved Open Questions	OQ-N (pending TD + inventory ## Open questions)
### UI Coverage Gaps	UIG-N (cap tem TD mas inventory sem verb; so se UI populated)
### Custom rule findings	CSF-N / CIE-N / ... (previões dos rules; so se rule rodou)
### Capability Consistency	CC-N (slicing only - typo em covers.capabilities; HARD-ERROR)
_(Cada secão renderiza _None_ se vazia. Categoria sumida = N/A naquele modo.)_	
## Cross-slice Advisories	MC-cross-N (phase + slicing only; NAO flipa status)
## Active Suppressions	(custom rules honraram entries de suppressions.md)
## Resolved Issues	(audit trail - preservado para sempre)
- **MD-1** _(resolved_by {slug}/TD-11)_ - <summary>.	(NUNCA removido, NUNCA renumerado)
- ...	
_(ou _No issues resolved yet_ na primeira run)_	

Matriz dos 8 checks — phase/task + flip status

ID	Phase	Task	Flip	Quando dispara
IC-N	X	X	dirty	Escopo ↔ TD contradicao; verb UI cita cap fora; Frontend TD orfa (UI absent/ deferred & Renders in: ui-contracts)
AMB-N	X	X	dirty	Cap bullet (phase) ou prose scope (task) vago demais p/ derivar SIs
MD-N	X	X*	dirty	Cap sem TD (uncovered bullet phase-only); decisao sem TD; shared-types contract sync #29 (phase + UI active/ logic-only; never deferred)
DG-N	X	-	dirty	Cap pressupoe pre-req de phase anterior nao planejada (cross-phase OR within-phase ordering nao explicito)
ICC-N	X	X	dirty	Current TD contradiz Inherited Conventions ou Inherited Decisions Detail
OQ-N	X	X	dirty	TD com Status: pending; ## Open Questions from Inventory (verbatim, 1 por bullet)
UIG-N	X	X	dirty	Cap tem TD coverage MAS sem verb no inventory MAS nao esta em Non-UI/Deferred. Skip se UI absent/deferred/ logic-only (zero verbs by construction)
CC-N	X†	-	dirty	SLICING ONLY (≥2 slices p/ NN). covers.capabilities tem entry que nao bate verbatim em project-plan.md phase NN. BLOQUEIA Check 8.b ate fix.
MC-cross-N	X†	-	NO	SLICING ONLY. Phase cap NAO coberta por nenhuma slice. ADVISORY - separado, em advisories: array.
* task: so dispara se TD doc existe; sem doc = 'task-sem-research'				
† Check 8 suprimido em fase monolitica (count=1) Phase tem 8 checks + custom rules. Task tem 6 (sem DG, sem CC/MC-cross).				

Lifecycle do issue + verdict + fast path

Lifecyle de um issue (audit trail)	
run 1	run 2 (apos /plan-resolve)
MD-1: open	→ MD-1: resolved resolved_by: {slug}/TD-11
AMB-1: open	→ (some - resolved esta run ja foi compactado) AMB-2: open (novo, lowest-unused-N por categoria)
Invariantes: - 'resolved' NUNCA removido do frontmatter - IDs NUNCA renumerados - Body sempre regenerado from scratch - Mas Resolved Issues acumula sempre	
Verdict (status: clean dirty)	
status: clean iff TODO issue do frontmatter tem status: resolved	
status: dirty caso contrario	
Advisories (MC-cross-N) NUNCA flipam status. Custom rule findings flipam conforme severity declarada no rule. CC-N (hard-error slicing) flipa status: dirty E bloqueia Check 8.b ate ser resolvido.	
Fast path (task mode, 1a run apenas)	
validation.md nao existe AINDA AND technical-decisions-{slug}.md ausente AND zero IC, AMB, MD, ICC, OQ, UIG → status: clean direto → task vai pra /plan-build sem precisar de /research Util pra: 'add tests', 'refactor X per existing convention' - tasks que herdam TUDO da phase lineage. Se UM check disparar OU decisions doc existir, fast path NAO aplica - verdict normal computado.	
Rerun apos /plan-resolve (canonical)	
1. /plan-resolve edita context.md (TDs flipam pending → decided ou rev appended) 2. /plan-validate roda again 3. issues que viraram 'resolved' no frontmatter sao dropped do body 4. novos checks podem surgir do proprio resolve (e.g., novo TD bate em Inherited Convention) 5. Loop ate status: clean 6. /plan-build libera	

Skill de resolução de problemas do contexto

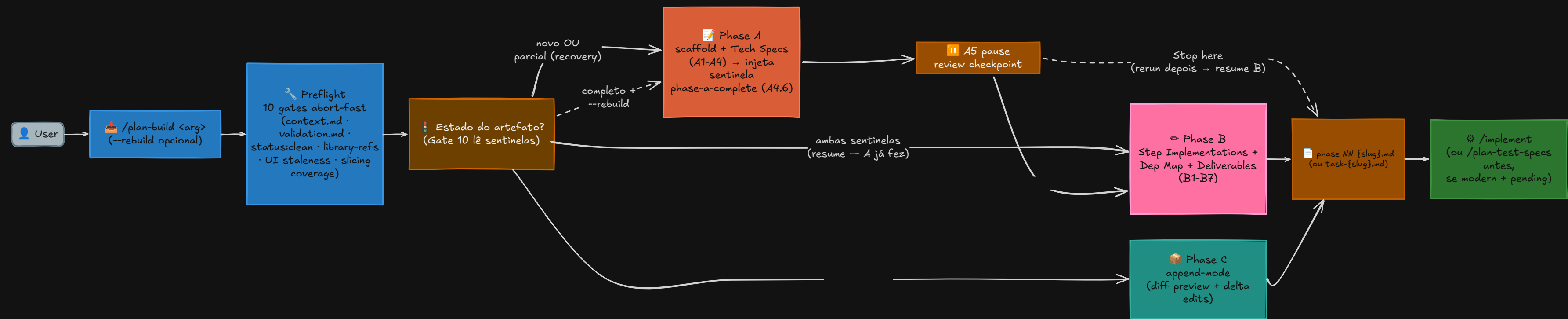


Anatomia — as 4 mutacoes que /plan-resolve faz no disco

plan-resolve eh o UNICO writer destes 4 arquivos (alem de project-plan.md que NUNCA toca):

[1] docs/decisions/technical-decisions-{slug}.md
<

Skill de planejamento das tarefas de implementação



/plan-build — o que cada stage faz (A1-A5, B1-B7, C1-C7)

Phase A — scaffold + Technical Specifications → pausa em A5 pra review

A1 Read context.md (única full read - memória pelo run)	A2 TD Scope filter Backend / Frontend / Cross-layer / Repo-wide	A3 Write scaffold (frontmatter + headers + 2 sentinelas + placeholders)	A4 Tech Specs (Data Model · API · UI Contracts · etc) incremental Edits	A4.5 Custom rules docs/rules/plan-build/* (podem abortar com ccr-pending sentinel)	A4.6 phase-a-complete sentinela injetada (prova que A passou A4.5 sem aborto)
--	---	---	--	---	--

↓ ↓ ↓ ↓ ↓ ↓

II

A5 — PAUSE (review checkpoint)

AskUserQuestion: "Continue now" → segue pra Phase B na mesma session · "Stop here" → encerra; rerun /plan-build re-entra direto em Phase B via Gate 10

Phase B — Step Implementations + closure → next-command pro /implement

B1 Reload context.md (same-session: skip - msg log; new: re-read)	B2 Decompose em SIs (capabilities + verbs do inventory)	B3 Validate deps (DAG sem ciclos + lib coverage per-SI)	B4 SI-by-SI Edits NO-LOOKBACK (substitui sentinela 1 na 1a SI)	B5 Dependency Map (DAG SI → SI consolidado)	B6 Deliverables (consolidado por subprojeto)	B7 Next-command legacy: /implement modern: /plan-test-specs (se pending placeholders)
---	---	---	---	---	--	--

Phase C — append-mode (rota alternativa quando o plano já está completo + context.md mudou)

C1 Enumerate deltas (context.md mudou desde build? quais novas/amendments?)	C2 Classify cada delta Edit cirúrgica / append amendment / new behavior / annotate superseded	C3 Diff preview MANDATORY (AskUserQuestion com summary; cancela → abort)	C4 Apply edits (nunca renumera; nunca modifica SIs status: completed)	C5 sources_mtime refresh MANDATORY (senão próxima run replays mesmos deltas)	C6 progress.md update (rows pras novas SIs)	C7 Next-command legacy: /implement <new-SI> modern: /plan-test-specs (re-stamp + new)
--	---	--	--	---	---	---

Sentinelas — máquina de estados que Gate 10 lê pra decidir Phase A vs Phase B vs Phase C

As 3 sentinelas (HTML comments) vivem no artefato e indicam em que estado o build está:

←!— SIs will be written in Phase B →

←!— phase-a-complete →

←!— {rule-id}-pending →

SENTINELA 1

SENTINELA 2

SENTINELA 3

escrita por A3 (scaffold)

escrita por A4.6 (final de A)

escrita por A4.5 em rule abort

· removida por B4 (1a SI)

· removida por B7 (build complete)

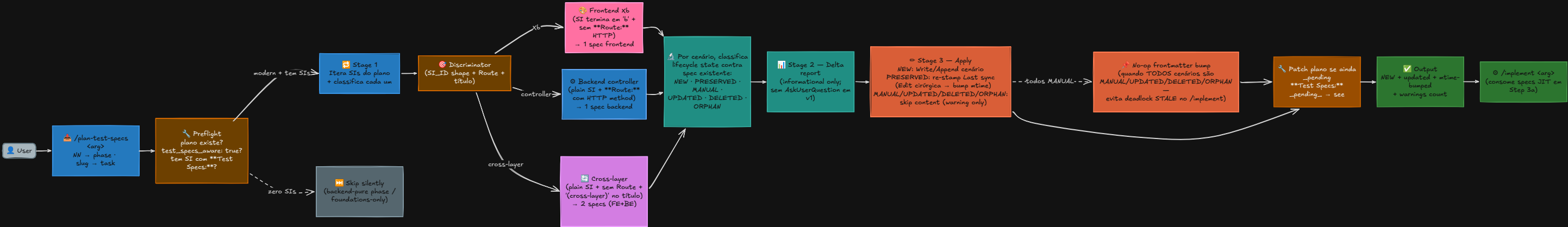
· canonical: ←!— ccr-pending →

Gate 10 lê com 1 grep e decide a rota:

sentinela 1	sentinela 2	Significado	→ Rota
ausente	ausente	arquivo não existe OU B7 completou	→ fresh A ou Phase C (append)
presente	presente	A terminou clean, B ainda não começou	→ resume Phase B
presente	ausente	A errou mid-A4 ou rule A4.5 abortou (parcial)	→ fresh A (overwrite)
ausente	presente	B começou (substituiu sentinela 1) e errou	→ fresh A (overwrite)

Resumo: sentinela 1 prova "A escreveu scaffold"; sentinela 2 prova "A terminou clean"; ausência das duas = ou fresh start, ou plano completo (rota Phase C).

Skill de planejamento de testes end-to-end



/plan-test-specs — 3 stages · 3 tipos de SI · 6 lifecycle states

Pipeline — 3 stages (Stage 1 classifica · Stage 2 reporta · Stage 3 aplica)

Stage 1 — Classify Itera cada SI do plano via Grep '^### SI-' + bounded read. Classifica via discriminator (SI_ID shape + **Route:** field + '(cross-layer)' keyword no título). Para cada cenário existente nos specs, classifica lifecycle state (NEW / PRESERVED / MANUAL / UPDATED / DELETED / ORPHAN).	Stage 2 — Delta report Emite UM bloco informational (sem AskUserQuestion em v1): - Per-spec breakdown: NEW · PRESERVED · UPDATED · MANUAL counts - Coverage gate (ACs sem cenário) - Warnings com resoluções acionáveis - Apply pass preview (N NEW + M re-stamps)	Stage 3 — Apply Para cada spec: NEW → Write/Append cenário PRESERVED → re-stamp Last sync (Edit cirúrgica) outros → skip content Patch **Test Specs:** _pending no plano → see '<path>'. No-op frontmatter bump se TODOS cenários forem MANUAL (evita deadlock STALE no /implement).
--	---	---

Discriminator — os 3 tipos de SI viram specs (e o que define cada um)

Frontend Xb Sufixo 'b' no SI_ID (ex: SI-03.5b) **Test Specs:** SIM **Route:** AUSENTE (ou Route sem HTTP method - Xa tem '/signup', não 'POST /signup') → emite 1 spec frontend: <fe-subproject>/specs/<feature>.plan.md runner: playwright Setup: tests/fixtures.ts Voice: 'usuário clica em X'	Backend controller Plain SI_ID sem letra (ex: SI-03.3 ou SI-03.3.1) **Test Specs:** SIM **Route:** PRESENTE com HTTP method prefix (regex: **Route:** (GET POST PUT PATCH DELETE) /) → emite 1 spec backend: <be-subproject>/specs/<feature>.plan.md runner: jest+supertest pytest gotest ... Setup: beforeEach truncate Voice: 'POST /auth/register'	Cross-layer Plain SI_ID sem letra + '(cross-layer)' no título (byte-verbatim, lowercase, parênteses) **Test Specs:** SIM **Route:** AUSENTE → emite 2 specs (FE + BE): <fe-subproject>/specs/ ... <be-subproject>/specs/ ... Coverage gate: warning só se algum AC fica uncovered em AMBOS os specs (não em qualquer um).
--	--	---

Lifecycle states — classificação por cenário (decide o que Stage 3 faz)

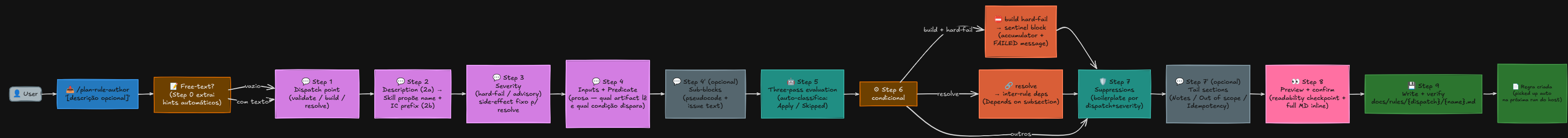
NEW AC do plano sem cenário cobrindo. Stage 3: gera cenário, marca Source: auto, set Last sync ao timestamp da operação.	PRESERVED Cenário existe; plan_mtime ≤ Last sync. Stage 3: re-stamp Last sync (Edit cirúrgica) - só pra bumpar file mtime.	MANUAL Source: manual OR Source field ausente. Stage 3: SKIP silently - user é dono do conteúdo.	UPDATED Source: auto AND plan_mtime > Last sync. Stage 3: warning only (v1) - user edita à mão. v2: --reconcile.	DELETED Source: auto cobrindo só ACs sumidas do plano. Stage 3: warning only (v1). v2: auto-delete do cenário.	ORPHAN Covers AC: #N aponta AC inexistente no SI atual. Stage 3: warning only (v1) - user corrige ou remove.
--	--	--	--	---	--

Os 2 bumps de file mtime que tornam a sequência canônica funcional

PRESERVED re-stamp Por que existe: /implement preflight checka file mtime (stat -c %Y) vs plan_mtime. Se PRESERVED skipasse silently, file_mtime ficaria velho → /implement abortaria STALE falsamente. Mecanismo: Edit cirúrgica substitui APENAS a linha **Last sync:** <old-iso> → **Last sync:** <current-iso> Conteúdo do cenário fica intacto. O Edit naturalmente atualiza o file_mtime. Resultado: Sequência canônica funciona - /plan-build (append) → /plan-test-specs → /implement	All-MANUAL no-op bump Por que existe: Quando TODOS os cenários estão em MANUAL/UPDATED/DELETED/ ORPHAN, ZERO Edits content-bearing disparam. File_mtime fica velho → /implement aborta STALE → user roda /plan-test-specs → MANUAL skip silently → STALE de novo. DEADLOCK INFINITO. Mecanismo: No-op Edit no frontmatter - substituição idempotente do valor atual. Para spec frontend, substitui 'subproject: frontend' por ele mesmo. Bumpa_mtime sem tocar nenhum cenário. NAO viola invariant 'user is owner of MANUAL content': só metadata muda, corpo dos cenários intocado.
---	---

Como o **Test Specs:** field flui no plano (trigger + populate)

1. /plan-build emite no SI:	**Test Specs:** _pending /plan-test-specs_ (placeholder; trigger pra esta skill processar o SI)
2. /plan-test-specs detecta + gera/re-stampa spec(s) + Edit cirúrgica patcheia a linha do plano: single-subproject: cross-layer:	**Test Specs:** see '<subproject>/specs/<feat>.plan.md' **Test Specs:** see '<fe-path>', '<be-path>'
3. /implement Step 3a lê **Test Specs:** populated + frontend: backend:	carrega spec → LLM autora arquivo de teste single-pass: tests/e2e/<feat>.spec.ts (1 spec + 1 arquivo, N cenários + N test() blocks) test/<feat>.e2e-spec.ts
4. Se /plan-test-specs rerun: detecta 'see ...' (já populated) + skip patch. NEW cenários ainda detectados normalmente.	



Em uma analogia — pense em ESLint, mas para o pipeline de planejamento

Voce ja conhece linters (ESLint, PyLint, RuboCop). Eles rodam automaticamente sobre seu CODIGO e flagam coisas erradas. Algumas rules sao built-in (e.g., 'no-unused-vars'); outras voce mesmo escreve pra checar regras do SEU projeto (e.g., 'use sempre nosso helper de Logger, nunca console.log').

Aqui é a mesma ideia — mas o 'codigo' que esta sendo lintado sao os ARTEFATOS DE PLANEJAMENTO:

- decisions docs em docs/decisions/*.md
- context.md (consolidado da fase)
- validation.md (estado das checks)
- phase-WN(slug).md / task-(slug).md (o plano final)
- openapi.json, screen-inventory, library-refs.md, etc.

Quando voce roda um dos 3 comandos do pipeline (/plan-validate, /plan-build, /plan-resolve), eles automaticamente buscam regras customizadas em docs/rules/(nome-do-comando)/ e rodam todas elas. A skill /plan-rule-author CRIA UM DESSES ARQUIVOS DE REGRA por voce, perguntando o que voce quer checar e gerando o markdown na forma certa pra ser auto-descoberto pelos 3 comandos.

3 momentos onde sua regra pode rodar (escolha 1)

plan-validate
(antes do build)

Quando rodar:
Voce roda /plan-validate
Pra conferir se o plano esta coerente, antes de autorizar /plan-build.

Use pra detectar:
problemas em arquivos que ja existem no disco - decisions docs, openapi.json, inventory.

O que sua regra faz:
encontra problema -> entra na lista de issues -> user ve, corrige, re-valida.

Exemplo:
openapi.json tem endpoint
POST /payments mas nenhuma decisions doc menciona pagamentos.

plan-build
(durante o build)

Quando rodar:
Voce roda /plan-build
Pra gerar o arquivo final do plano (Tech Specs + UI).

Use pra detectar:
problemas que so aparecem DEPOIS que o plan e renderizado - Algo no texto recom-escrito que nao bata com uma fonte de verdade.

O que sua regra faz:
encontra problema -> aborta build, deixa marca no arquivo, user resolve e tenta de novo.

Exemplo:
API Contract menciona lib redis mas redis nao esta no library-refs.md

plan-resolve
(corrige automatico)

Quando rodar:
Voce roda /plan-resolve
DEPOIS que validate ou build flagou algo.

Use pra detectar:
nada - esta dispatch no check. Ela CONSEGUE. Edita arquivos direto.

O que sua regra faz:
detecta uma situacao especifica -> edita decisions doc / context.md / library-refs.md automaticamente.

Exemplo:
quando uma TD muda de QUIT to, update um
=>Revisions=> block com motivo'.

Walkthrough com exemplo real — vamos criar uma rule pra detectar 'openapi drift'

Cenario: voce quer que /plan-validate FLAGUE quando openapi.json tem endpoints que nao estao mencionados em nenhuma decisions doc.

USER: /plan-rule-author 'detectar openapi.json com endpoints nao mencionados nas decisions'

• Step 0 (auto) - Skill leu seu texto e ja extraiu:
dispatch: validate severity: nao explicito (vai perguntar)
descricao: 'openapi.json com endpoints nao mencionados nas decisions'

• Step 1 (Skill ja sabe - skipped)

• Step 2a - Skill confirma: 'Capturei a descricao como X. Mudar antes de prosseguir?'

USER: 'tudo certo'

• Step 2b - Skill PROPOE nome derivado:
Filename: openapi-orphan-endpoints.md
IC prefix: OPENAPI-ORPHAN-ENDPOINTS-N
Rationale: clusters com sibling rules de openapi-> (se existirem)
USER: 'aceitar'

• Step 3 - Skill pergunta severity: USER escolhe 'hard-fail' (Queremos BLOQUEAR o build)

• Step 4 - Skill pergunta Inputs + Predicate:
USER: Inputs: nextjs:project/openapi.json + todos docs/decisions/*.md
Predicate: pra cada endpoint X em openapi.json, se NENHUM decisions doc mencionar 'X' ou qualquer ==Recommendation== ou ==Trigger==, fire issue.

• Step 4' (optional) USER: 'add issue text literal'

• Step 5 (auto) - Skill propoe Three-pass evaluation: APPLY (pode haver mediation TD futura)
USER: 'ok'

• Step 6 - N/A (e' validate, nao build)

• Step 7 (auto) - Skill injeta boilerplate de Suppressions

• Step 7' (optional) USER skipa todos

• Step 8 - Skill mostra PREVIEW do MD inteiro.
USER: 'sim' (escreve!)

• Step 9 - Skill escreve docs/rules/plan-validate/openapi-orphan-endpoints.md
+ diz: 'Pronto. Roda /plan-validate fase-X pra ver a rule disparar.'

PROXIMA vez que voce rodar /plan-validate, a rule e' auto-descoberta. Sem editar codigo nenhum.

O arquivo Final, anotado — cada parte saiu de qual Step

```
---
description: "openapi.json com endpoints nao mencionados nas decisions"  ← Step 2a (DESCRIPTION)
status: active                                                           ← default ('disabled' = skill-readable mas inerte)
---

# OpenAPI Orphan Endpoints                                              ← Step 8 (HUMAN_TITLE = title-case, NUNCA kebab)

Esta rule corre durante /plan-validate (apos as checks built-in 1-8)    ← Step 8 (OPENING_PARAGRAPH =
e bloqueia o build se openapi.json tiver endpoints que nao aparecem     2-3 frases plain: O QUE +
em nenhum decisions doc. Se encontra, vai pra validation.md como issue. QUANDO + ACAO)

## Emission                                                             ← Step 7 (boilerplate auto)
- **Prefix:** OPENAPI-ORPHAN-ENDPOINTS-N (one IC per matching condition)
- **Severity:** hard-fail - flips validation.md status: dirty
- **Destination:** validation.md issues: array

## Inputs                                                               ← Step 4 (INPUTS_PROSE verbatim)
- nextjs:project/openapi.json (full read)
- Todos arquivos em docs/decisions/*.md (Recommendation + Trigger fields)

## Predicate                                                            ← Step 4 (PREDICATE_PROSE verbatim)
Para cada endpoint (method + path) em openapi.json, busca o texto em
qualquer ==Recommendation== ou ==Trigger== de qualquer decisions doc.
Se nenhum doc menciona, accumulate findings.

### Issue Text                                                         ← Step 4' (opt-in)
---
OPENAPI-ORPHAN-ENDPOINTS-N - Endpoint [METHOD] [PATH] em openapi.json
sem cobertura em decisions docs. Adicione TD via /research <slug>
---

## Three-pass evaluation                                               ← Step 5 (auto: Apply)
Para cada candidato finding:
  Pass 0: openapi-runtime cobre? skip se sim
  Pass 1: mediation TD cobre? skip se sim
  Pass 2: SI no plan file cobre? skip se sim
  Pass 3: nenhum captura - fire OPENAPI-ORPHAN-ENDPOINTS-N

## Suppressions                                                         ← Step 7 (boilerplate auto)
1. Le (target_dir)/suppressions.md se existir: no-op silently se ausente.
2. Para cada entry com id 'OPENAPI-ORPHAN-ENDPOINTS-N', adiciona ao set
3. Filtra emissões; honored IDs vao pra ## Active Suppressions de validation.md.
```

3 coisas pra lembrar

Zero edição de código

Voce nao precisa editar as skills built-in pra adicionar uma rule.

O host (plan-validate /plan-build /plan-resolve) faz Git na pasta docs/rules(dispatch/) e carrega TUDO que encontrar lá.

Criar a rule = criar o MD. Pronto.

status: active / disabled

Voce pode 'desligar' uma rule sem deletar.

No frontmatter:
status: disabled

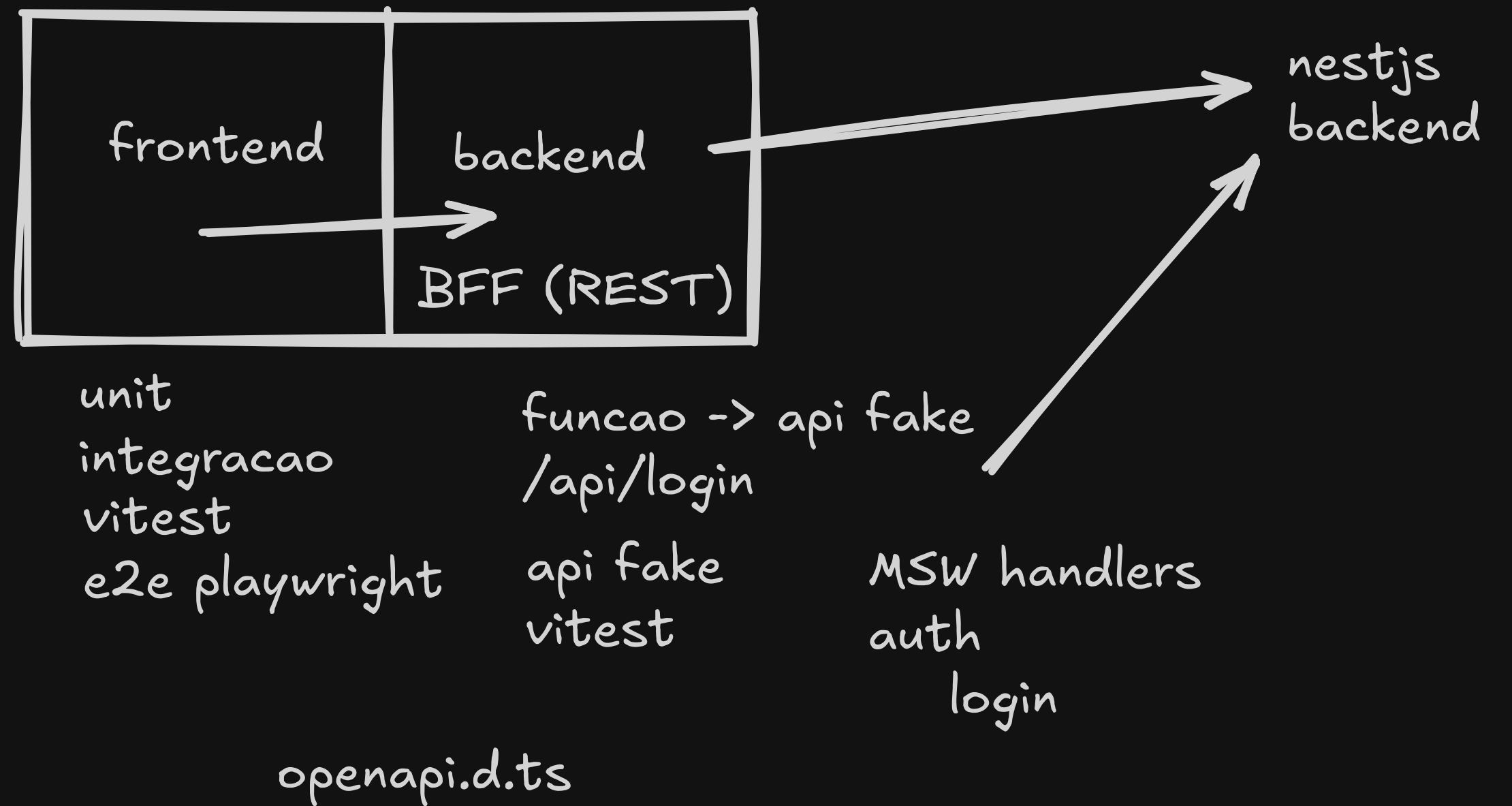
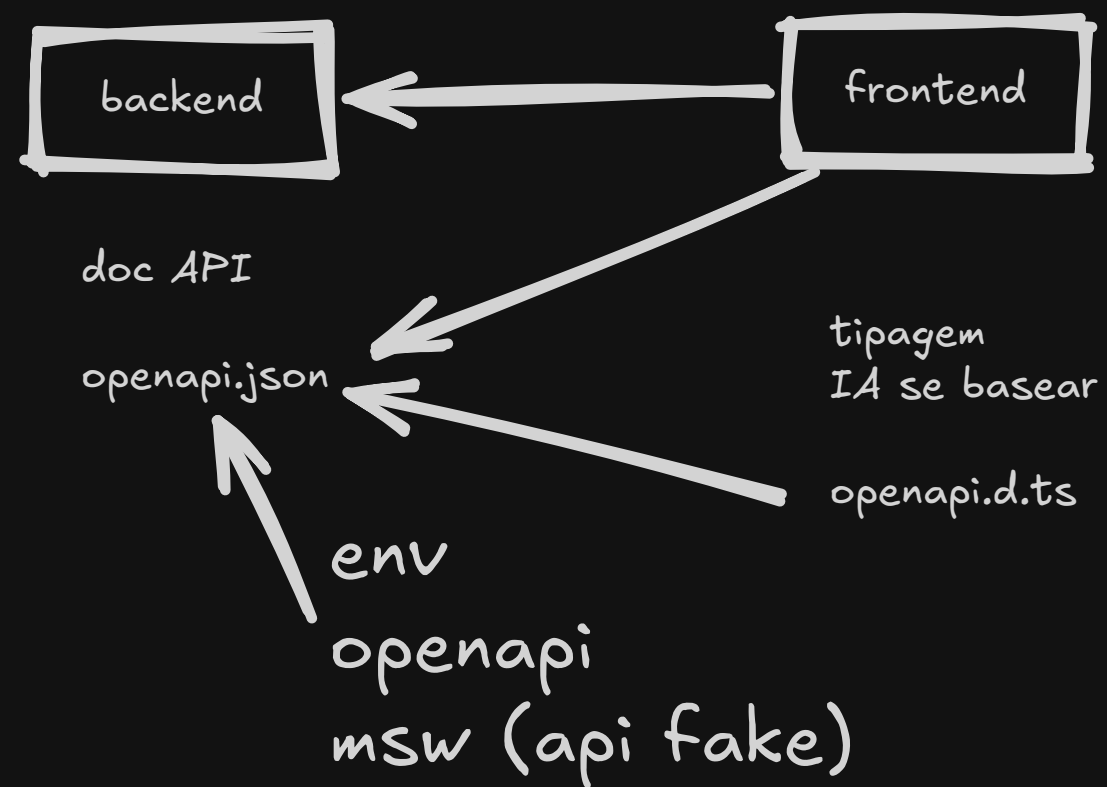
A skill host vai LER o frontmatter, ver que esta disabled, e SKIP silently.

Util pra rules experimentais ou durante migração.

Suppressions por scope

Cada fase / task tem seu suppressions.md proprio:
docs/phases/phase-WN-X/
suppressions.md

Voce lista IDs especificos (OPENAPI-ORPHAN-N) pra suprimir naquela scope. Aparecem em ## Active Suppressions de validation.md (audit).



Próximos passos

- Pratique:
 - Implemente as próximas fases
 - Corrija detalhes, faça melhorias nas fases já existentes
 - Melhore o design do figma, construa telas e ajuste os DS tokens
- Use as skills no seu dia-a-dia
- Estruture seu workflow baseado etapas e feedbacks
- Sempre planeje mais que codifique